Theory of Computation

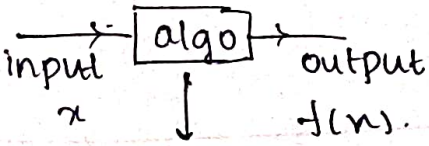
Mid - 25
End - 50
(3) SQ - 20
assign - 5

↙
carried out by executing programs.

↳ A program is an algorithm expressed in some programming language.

ToC ⊆ ToA - - - - - - - -> Algorithm

→ An algorithm is a receipe for carrying out input to output transformation.



tells us how to compute a fkn

* $f : D \to R$ → Range of outputs of f
  - D: domain of inputs of f

* A fkn can have any number of algos

Eg: $isPrime : \mathbb{N} \to \{Yes, No\}$

$$isPrime(n) = \begin{cases} Yes & \text{if } n \text{ is prime} \\ No & \text{if } n \text{ is not a prime} \end{cases}$$

Goal: To identify which fkns admit algo to compute them.

{ for fkn f there is an algo to compute f

{ rest do not admit any algorithm to compute them

*Set Membership problem (SMP):

S is a set

Given any a, to decide if $a \in S$ ⟶ SMP

$f: D \to R$ $\quad graph(f) = \{(a,b) \mid f(a) = b\}$

↙

Subset of $D \times R$

→ Suppose we show there is no algo to solve SMP for graph(f), then we can conclude there is no algo to compute f

If

$$b = f(a) \iff (a,b) \in graph(f)$$

→ Symbol: 0, 1, 2 → 9, a, b, z, α

Alphabet: A set of distinct symbols

↓

$\Sigma = \{0,1\}$

finite ↙ $\Sigma = \{a,b,c\}$

String: If $\Sigma = \{a,b\}$, then strings over Σ are

↓

w

a, ab, abab, ba, aabb. - -

length: If w is a string, then length of w = |w|.

|abc| = 3

→ Empty string has length 0 & represented by ϵ (Epsilon)

If w = xyz & v = abc

wv = xyzabc

→ if $a \in \Sigma$, then a^n denotes n a's concatenated

$a^5 = aaaaa$, a^0 : defn $a^{n+1} = a^n \cdot a$

→ Reverse : If w is string then w^R → reverse

* A substring Y of a string w such is a string such that

$w = xyz$ with $|x|, |y|, |z| \geq 0$ and $|x| + |y| + |z| = |w|$

* Σ^* → infinite set that denotes all finite strings over Σ.

Eg: $\Sigma = \{0, 1\}$. $\Sigma^* = \{\epsilon, 0, 1, 01, 00, 11, 0100 \dots\}$

$\Sigma^0 = \{\epsilon\}$

$\Sigma^n = \{w \mid w = xy$ such that $x \in \Sigma^{n-1}$ & $y \in \Sigma\}$.

$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \dots \cup \Sigma^n \cup \dots = \bigcup_{i=0}^{\infty} \Sigma^i$

if $\phi = \{\}$ — $\phi^* = \{\epsilon\}$. * → called as kleen closure.

* Language

→ A language or formal language over Σ is any subset of Σ^*.

Ex: $\Sigma = \{0, 1\}$ $L = \{01, 11001, 1001, 10101110\} \subseteq \Sigma^*$.

→ Language can be finite or infinite → when L → is Σ^* because $\Sigma^* \subseteq \Sigma^*$

Eg: $L_1 = \{x \in \{0, 1\}^* \mid x$ has even no. of 0's & even no. of 1's$\}$

$\epsilon, 00, 0011, 11, \dots$

* $L = \{w \mid w = w^R\}$ → palindromes.

* complement of L

$\bar{L} = \Sigma^* - L$

* $L_1 \cdot L_2 = \{xy \mid x \in L_1, y \in L_2\}$

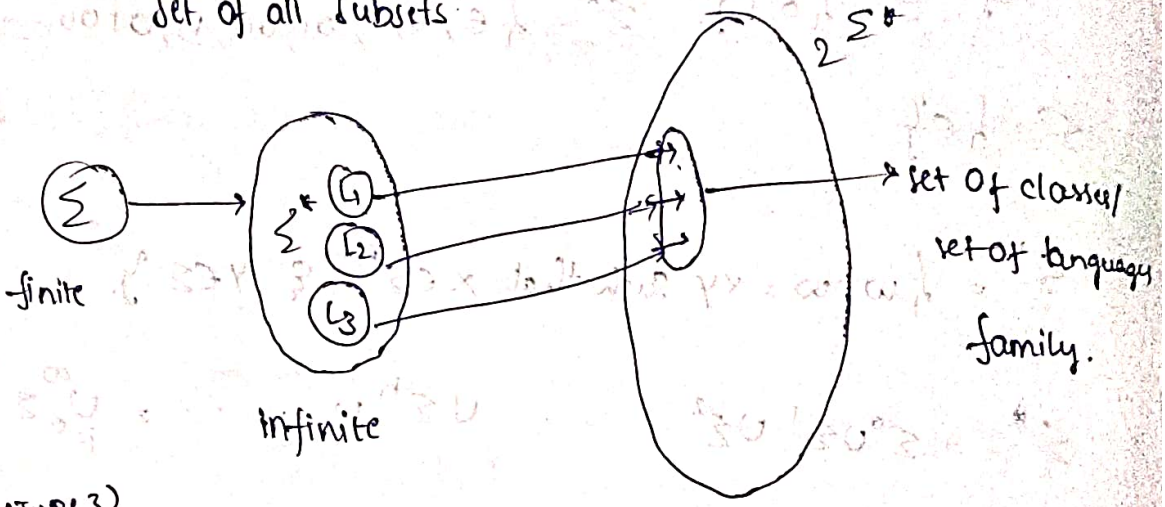
$L^0 = \{\epsilon\}$

$L^* = \bigcup_{i=0}^{\infty} L^i$

$L^+ = L^* - \{\epsilon\} = \bigcup_{i=1}^{\infty} L^i$

* The power set of Σ^* is represented by 2^{Σ^*}

(set of all subsets)



(Type 3)

* Regular languages → finite automata

(Type 2) Context free languages → push down automata

(Type 1) Context sensitive languages → linear bound automata

(Type 0) Recursively enumerable languages → Turing machine.

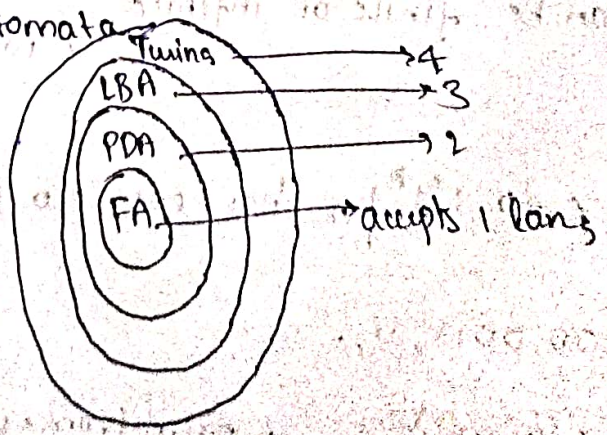
Automata

↓

A mathematical model that helps to check whether a string belongs to a language or not.

* Expressive power of automata

chomsky Hierarchy



* Finite automata / FSM / FSA

FS → finite state
M → machine
A → automata

Eg. $\Sigma = \{0,1\}$

$L = \{x \in \{0,1\}^* \mid x$ has even no. of 0's$\}$

$L = \{\epsilon, 00, 001, 100, 010, \ldots\}$

* if string length is x then it will pass x+1 states

→ Deterministic finite automata (DFA) until unless specified as non deterministic finite automata

5 tuple machine
$[Q, \Sigma, \delta, q_0, F]$
sets.

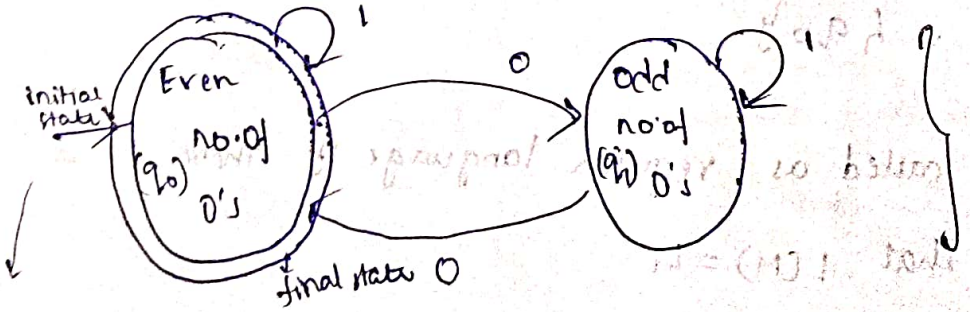
0 1 1 0 1 1 0 0

ε 0 1 1 0 1 1 1 0 0
↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑
even odd odd odd even even even even odd even

* no. of edges = no. of states * no. of input alphabets

ε 1 0 1 1 0 0 1
1 ↑ ↑ ↑ ↑ ↑ ↑ ↑
E E O O O E O O

$\notin L$ (Odd no. of zeroes)



deterministic finite automata

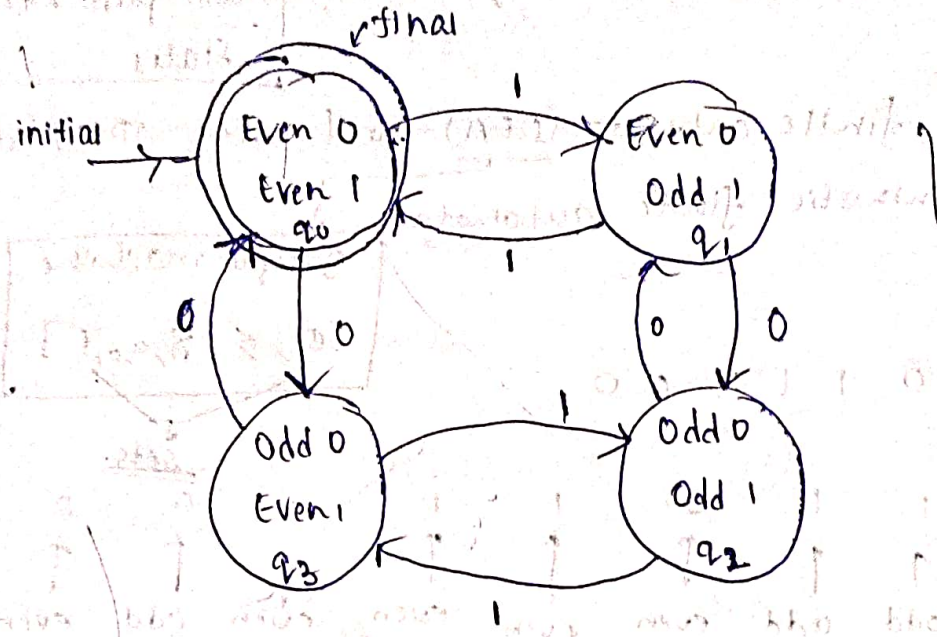
Start state represented as arrow from nowhere.

1) States : $Q = \{q_0, q_1\}$
2) Alphabet : $\Sigma = \{0,1\}$
* 3) Transition function : $\delta : Q \times \Sigma \to Q$
4) Initial state : $q_0 \in Q$; $q_0 = q_0$
5) Final state : $F \subseteq Q$; $F = \{q_0\}$ → for this case (example)

Current state	input alphabet	next state
q_0	0	q_1
q_0	1	q_0
q_1	0	q_0
q_1	1	q_1

Eg: $L_2 = \{x \in \{0,1\}^* \mid x$ has even no. of 0's and even no. of

e	0	1	1	0	1	1	0	0	1	0	1
↑	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑	↑
E 0	0	0	0	E	E	E	0	E	E	0	0
E 1	E	0	E	E	0	E	E	E	0	0	E



1) States : $Q = \{q_0, q_1, q_2, q_3\}$

2) Alphabet : $\Sigma = \{0, 1\}$

3) Transition rule $= \delta : Q \times \Sigma \longrightarrow Q$

4) initial state $q_0 = q_0$

5) final state $F = \{q_0\}$.

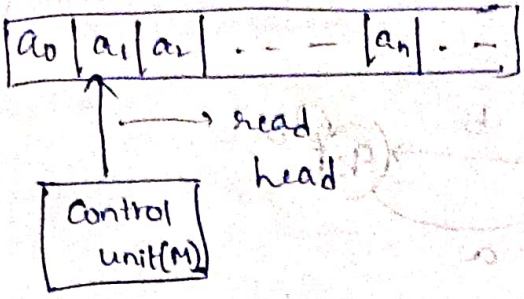
* For any given regular language there is only one DFA

* A language L_1 is called as regular language if there is a DFA M such that $L(M) = L_1$

↓

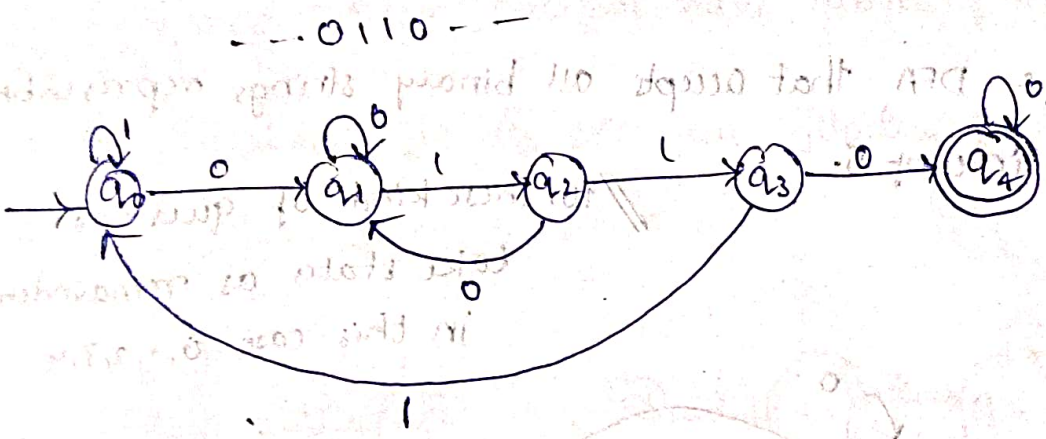
* unique.

$$L(M) = \{x \in L \mid \hat{\delta}(q_0, x) = q_f \in F\}$$



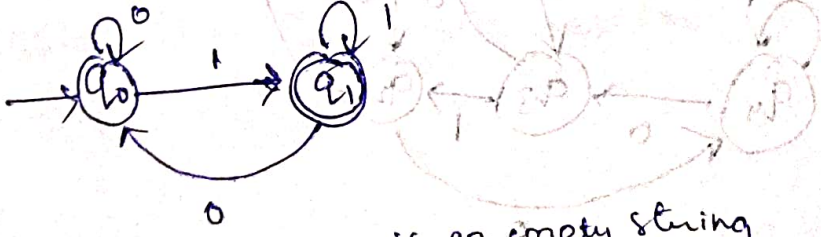
* The no. of state changes is one more than the number of symbols.

Eg: $L_3 = \{ x \in \{0,1\}^* \mid x$ has a substring 0110 $\}$.

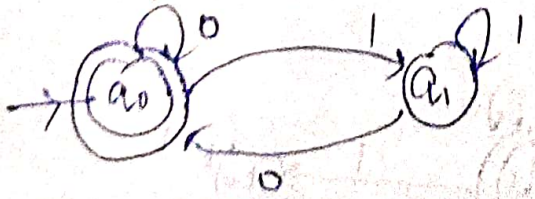


* 60% cases trap for minimal string other than ϵ

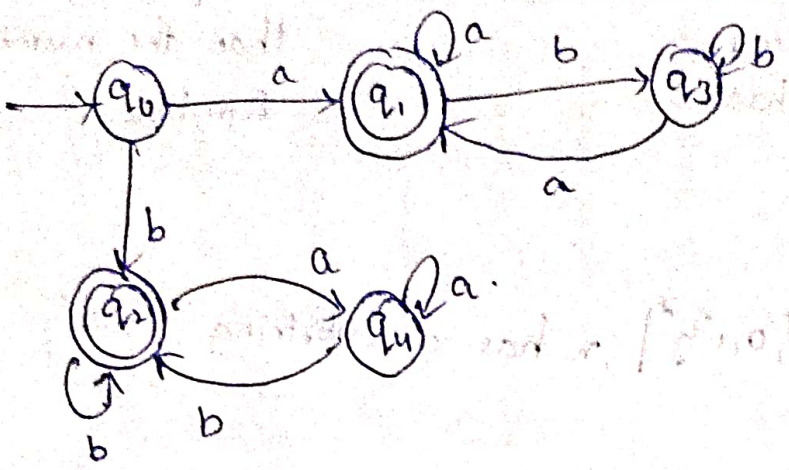
Eg: $L_4 = \{ x \in \{0,1\}^* \mid x$ ends with 1 $\}$.



Eg: $L_5 = \{ x \in \{0,1\}^* \mid x$ ~~has either zero number of 0's or~~ is an empty string ends with 0 $\}$.

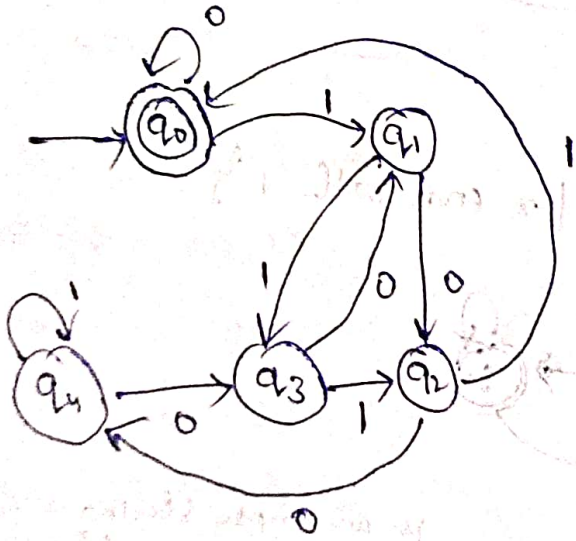


Eg: $L_6 = \{x \in \{a,b\}^* \mid x$ begin and end with the same char



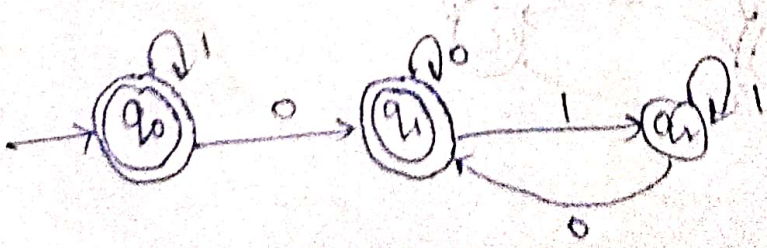
Eg: Design a DFA that accepts all binary strings representing a multiple of 5.

// For these kind of questions take states as remainders in this case 0, 1, 2, 3, 4



* can be applied for all bases

Eg: $L_7 = \{x \in \{0,1\}^* \mid x$ has either zero number of 0's or ends with 0 $\}$



* Operations on languages :

1) Compliment

if L is a regular language $\rightarrow \boxed{\bar{L} = \{x \in \Sigma^* \mid x \notin L\}} \Rightarrow \bar{L} = \Sigma^* - L$

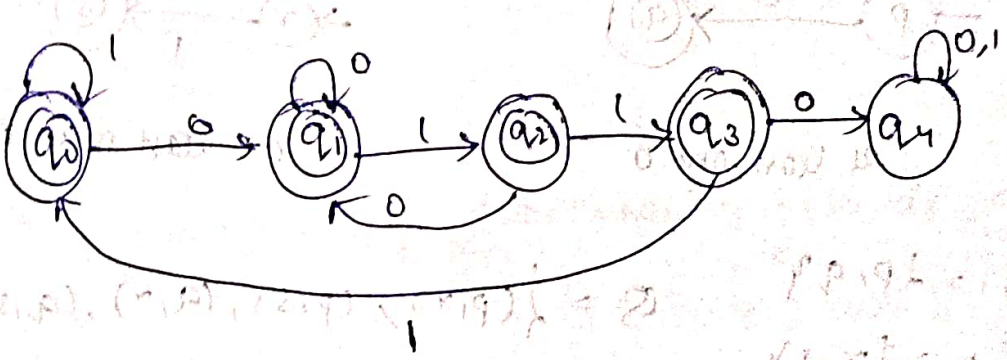
$\bar{L} = \Sigma^* \setminus L$

eg : $L_3 = \{x \in \{0,1\}^* \mid x$ has a substring $0110\}$.

$\bar{L_3} = \{x \in \{0,1\}^* \mid x$ does not have substring $0110\}$.

→ For complement in the DFA just flip the final & non-final states from L DFA.

$\bar{L_3}$



* if L is regular then $\bar{L}$ is also regular.

2) Union

$L_1, L_2 \subseteq \Sigma^*$

$L = L_1 \cup L_2$

if L_1 and L_2 are regular languages then L is a regular language

Since L_1 & L_2 are regular languages then there are two DFAs M_1 & M_2 accepting these languages respectively.

$M_1 = \{Q_1, \Sigma, \delta_1, q_{0_1}, F_1\}$

$M_2 = \{Q_2, \Sigma, \delta_2, q_{0_2}, F_2\}$

$M = \{Q, \Sigma, \delta, q_0, F\}$

$\rightarrow Q = Q_1 \times Q_2$

$Q = \{(p,q) \mid p \in Q_1 \text{ and } q \in Q_2\}$

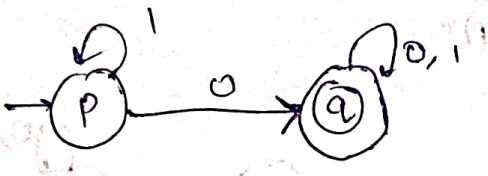
$\rightarrow \delta((p,q),a)) = (\delta(p,a), \delta(q,a))$

$\rightarrow q_0 = (q_{0_1}, q_{0_2})$

$\rightarrow F = \{(p,q) \mid p \in F_1$ ~~and~~ or $q \in F_2\}$.

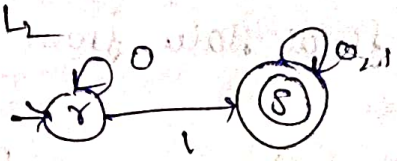
Eg:

L_1



at least one 0

L_2

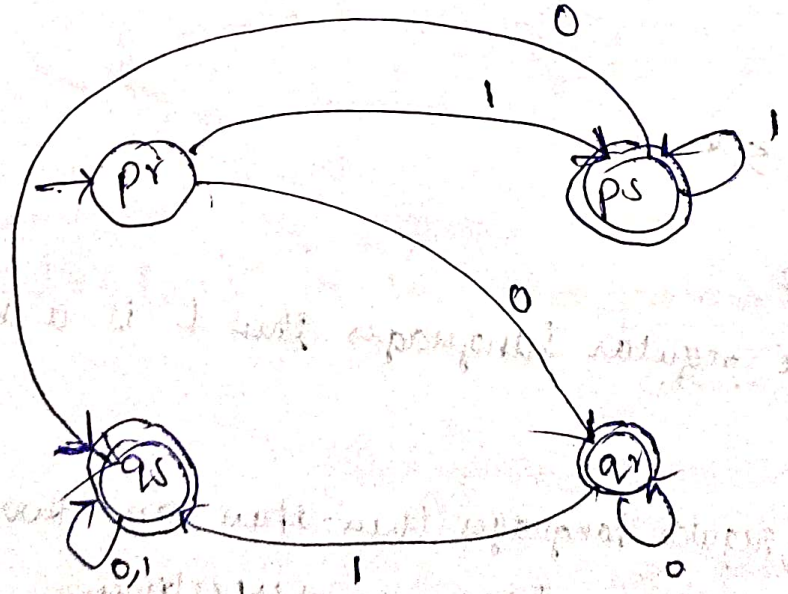


at least one 1

$Q_1 = \{p, q\}$

$Q_2 = \{r, s\}$

$Q = \{(p,r), (p,s), (q,r), (q,s)\}$.



$\delta((p,r),0) = (\delta(p,0), \delta(r,0))$

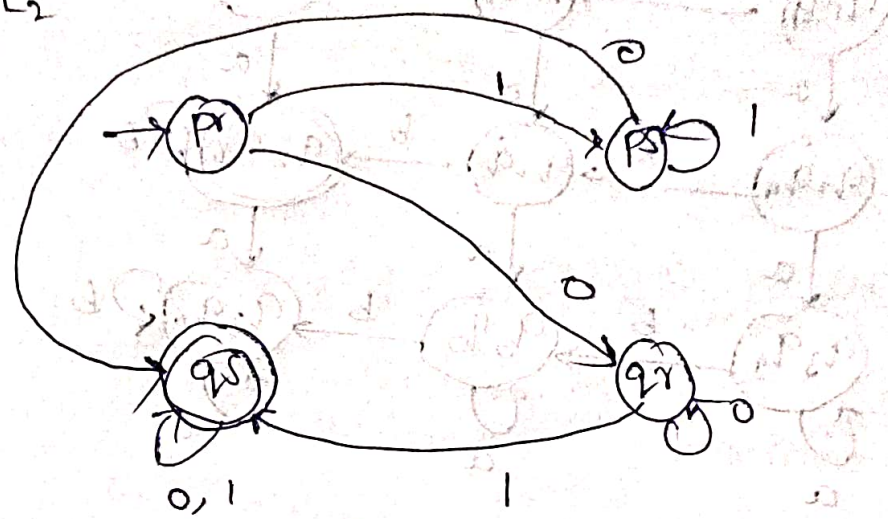
$= (q,r)$

3) Intersection

→ only final state is different from union.

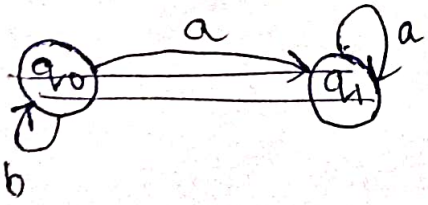
$$F = \{(p,q) \mid p \in F_1 \text{ and } q \in F_2\}$$

$L_1 \cap L_2$

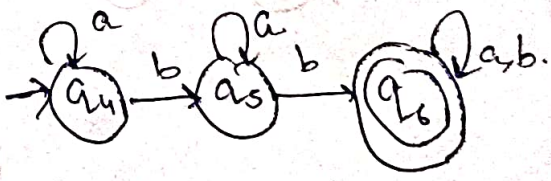


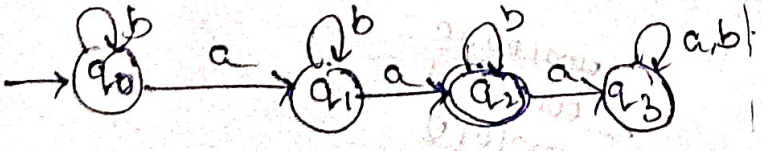
Eg: $L_8 = \{w \in \{a,b\}^* \mid w$ has exactly two a's and has atleast two b's

L_4 = w has exactly two a's

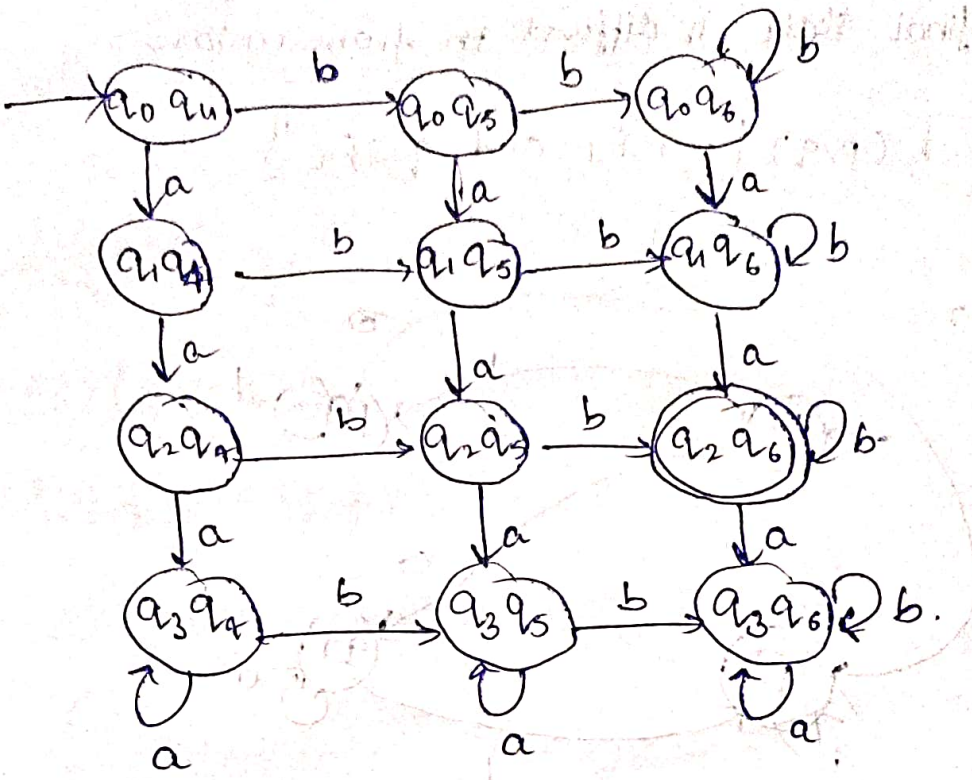


L_2 = w has atleast two b's





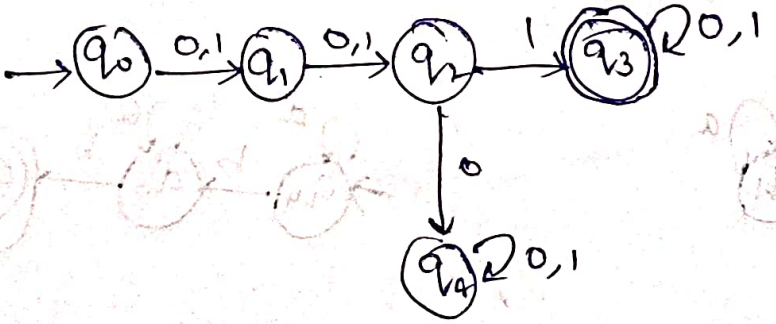
$L_1 \cap L_2$



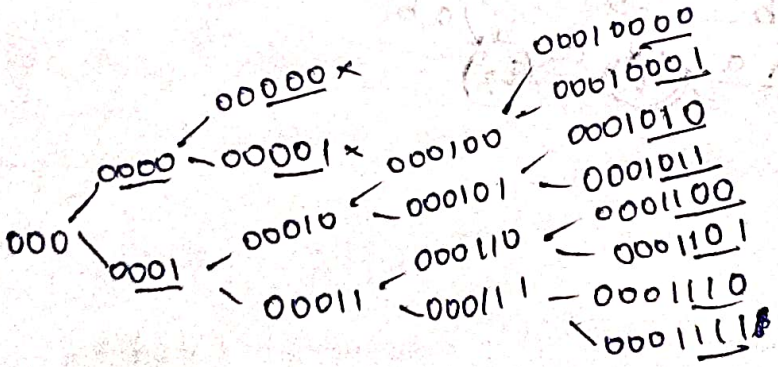
Eg: $L_{10} = \{x \in \{0,1\}^* \mid$ the third bit from the left end is 1 in $x\}$

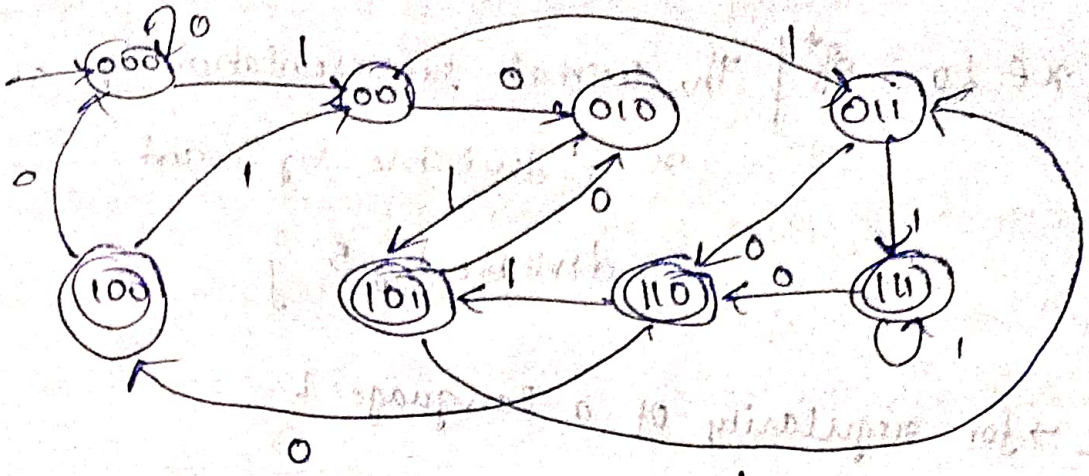
* $L_{11} = \{x \in \{0,1\}^* \mid$ the third bit from the right end is 1 in $x\}$

L_{10}



for L_{11}

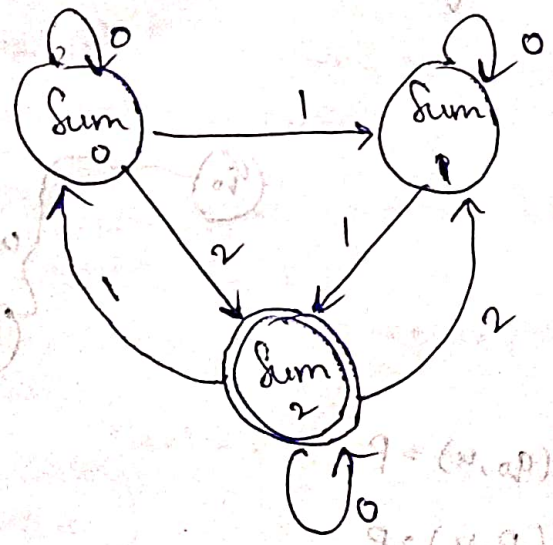




Eg: $L_q = \{x \in \{0,1,2\}^* \mid$ the sum of digits in x is $2 \bmod 3\}$

$21121101 \overset{=9}{\notin} L \rightarrow$ remainder 0

$21121112 \overset{=11}{\in} L \rightarrow$ remainder 2.



$2110100111222022 \in L.$

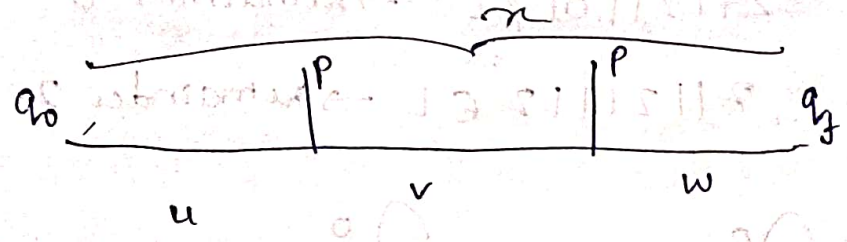
Eg: $L = \{ x \in \{0, \ldots, 9\}^* \mid$ The decimal representation of x is divisible by 3 and divisible by 5. $\}$

* Lemma → for regularity of a language L.

Assume L is regular, then it is accepted by a DFA.

Let that DFA has k states.

Imagine that a very long string (Eg: length of 2k) exists in L

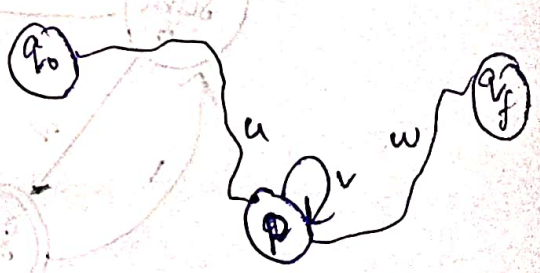


$\hat{\delta}(q_0, x) = q_f$

$\hat{\delta}(q_0, uvw) = q_f$

for string

δ → for symbol.



$\hat{\delta}(q_0, u) = p$

$\hat{\delta}(p, v) = p$

$\hat{\delta}(p, w) = q_f$

* If $uvw \in L$, then $uv^iw \in L$ for all $i \geq 0$ } → pumping lemma

uv^iw → pumping

→ if the resultant string is also in the L, then L is regular language.

* Pumping lemma: (Q on state & prove)

Statement:

Suppose L is regular then there exists a constant k that depends only on the language L such that for all $x \in L$, $|x| \geq k$ there exists u,v,w satisfying

1) $uvw = x$

2) $|uv| \leq k$, $|v| > 0$

3) $\forall i, i \geq 0$, $uv^i w \in L$

Eg: $L_{12} = \{x \in \{0,1\}^*, \text{~~$x \in$~~} x = 0^n 1^n, n \geq 0\}$ is not reg.

1) ~~Assume~~ the language is regular.

$x = 0^k 1^k$, $|x| = 2k \longrightarrow \geq k$.

$x = 000 \text{ —— } 0011 \text{ - - - —— } 11 = uvw$.

from condition(2) in P.L ——→ pumping lemma

we have

$|uv| \leq k$, $|v| > 0$.

which means

$uv = 000 \ldots\ldots 000$

Apply pumping for $i = 0$ for v^i in $uv^i w$

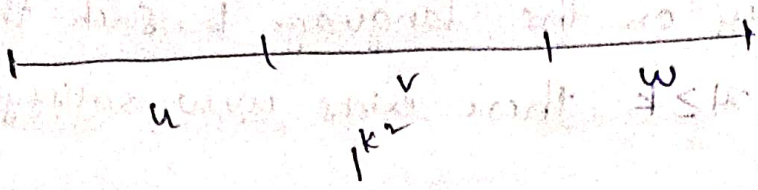
If we apply $i = 0$, then the remaining uw does not contain equal number of 0's and 1's. That means $uw \notin L$ or $uv^0 w \notin L$. But the lemma says $uv^i w \in L_{12}$. So we have arrived at a contradiction.

Therefore ~~L_{12}~~ the assumption is wrong and

L_{12} is not a regular language.

Eg: Prove that $L_{13} = \{x \in \{1\}^* \mid x = 1^{n^2}, n \geq 1\}$ is not regular

$x = 1^{k^2}$ $\quad |x| = k^2 \rightarrow \geq k$.



$|v| \neq 0 \quad |vw| \leq k$

$i \geq 2$

$uvvw \in L$

$|uvvw| = k^2$

$|uvvw| = k^2 + k$ ← max value of v ∵ $|uv| \leq k$

$(k+1)^2 > k^2 + k > k^2$ ⟶ Contradiction

↓ next string length

↓ curr string length.

∴ L_{13} not a regular language

k = 3

111 1111111
uv w

11 1 111 111
u v w

uv^2w

uuuuuuuu

$uv^2w \notin L$

Eg: Prove that the following languages are not regular

$L_{14} = \{0^i 1^j \mid i \neq j\}$

$L_{15} = \{a^m b^n \mid m < n\}$

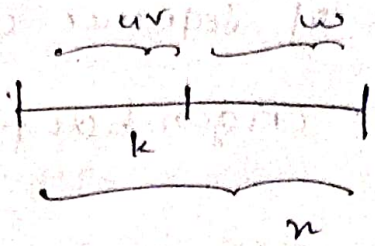
$L_{16} = \{a^m b^n \mid m > n\}$

$L_{18} = \{0^n 1^n 2^n \mid n \geq 0\}$

$L_{19} = \{www \mid w \in \{0,1\}^*\}$

$L_{20} = \{w = w^R \mid w \in \{0,1\}^*\}$

$L_{20} = \{ w = w^R \mid w \in \{0,1\}^* \}$.



$|uv| \leq k$.

$uv = (uv)^R = w$

$i = 0 \quad uv^0 = (uv^0)^R = w$

$u = u^R = w$

↙

not a palindrome because

$|w| > k$ and $|u| < k$.

Similarly for odd length $\underbrace{w}_{k} c \underbrace{w^R}_{k} \Rightarrow |wcw^R| = 2k+1$

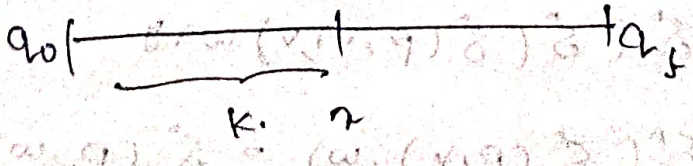
* Proof of Pumping Lemma (P.L):

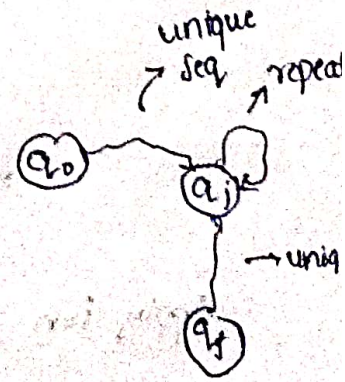
Since L is regular, there is a DFA M that accepts L

Let k be the number of states in M

Let $x \in L$; $|x| \geq k$

Consider the first k symbols of x.

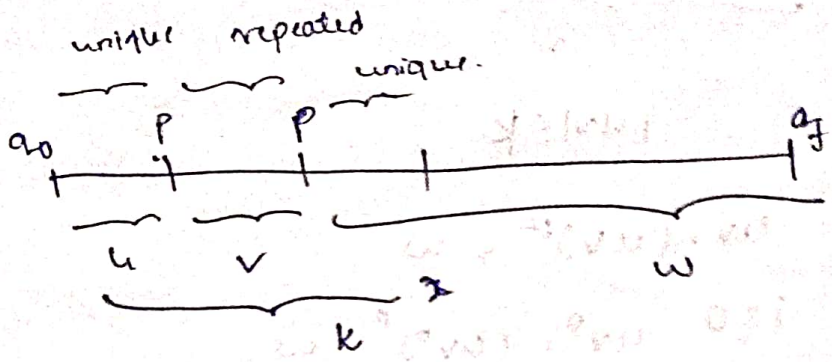




If $x \in L$, then M accepts x.

$$\boxed{\hat{\delta}(q_0, x) = q_f}$$

" Because there are only k states, and the first k+1 symbols require k+1 length of sequence of states. atleast 1 state is repeated ∴ (Pigeonhole principles).



$\delta^*(q_0, u) = p$

$\delta^*(p, v) = p$

$\delta^*(p, w) = q_f$

$\delta^*(q_0, uvw) = q_f$

Apply i=0 for $uv^iw \Rightarrow uv^0w \Rightarrow uw$

$\delta^*(q_0, uw) = \delta^*(\delta^*(q_0, u), w)$

$= \delta^*(p, w) = q_f$

Apply i = 2 for $uv^iw = uv^2w \Rightarrow uvvw$

$\delta^*(q_0, uvvw) = \delta^*(\delta^*(\delta^*(\delta^*(q_0, u), v)v)w)$

$= \delta^*(\delta^*(\delta^*(p, v), v)w)$

$= \delta^*(\delta^*(p, v), w) = \delta^*(p, w) = q_f$

Apply i=n for $uv^iw \Rightarrow uv^nw \Rightarrow uvv \cdots vw$

$\delta^*(q_0, uvv \cdots vw) = \delta^*(\delta^*(\delta^*(\delta^* \cdots \delta^*(q_0, u), v) \cdots w)$

first

$= \hat{\delta}(\hat{\delta}(\hat{\delta}(\hat{\delta}(\ldots \hat{\delta}(p,s),v),v),\ldots),w)$

$= \hat{\delta}(p,w) = q_f$.

There by using proof by induction the lemma is true.

Hence proved.

Eg: Prove

$L_{21} = \{a^i b^j c^k \mid i,j,k \geq 0,$ if $i=1$ then $j=k\}$

is not regular

* 1) To show that PL is true, then for all strings $x \in L$, such that $|x| \geq k$, x can be divided into uvw, satisfying the three conditions. Just showing it for one x is not enough.

* 2) To show that PL is false then choose one string $x \in L$, such that $|x| \geq k$. But now for every division of x into uvw, at least one of the three conditions is not satisfied.

* Conclusion:

L is regular $\Rightarrow A$

*

$(\exists k)(\forall x, x \in L, \wedge |x| \geq k)\ (\exists u,v,w)[x = uvw \wedge |uv| \leq k \wedge |v| > 0$

$\wedge (\forall i)\ uv^i w \in L]$

Prover		Verifier
Choose x	$\xleftarrow{L, k}$	Choose L, k
	$\xrightarrow{x}$	Choose uvw
Find i such that $uv^iw \notin L$	$\xleftarrow{uvw}$	
	$\xrightarrow{i, \text{result}}$	

* Non-deterministic Finite Automata (NFA)

→ Generalization of DFA.

→ Similar to DFA, only accepts regular languages.

→ DFA is deterministic

→ Every step of computation has unique transition from one state to another.

→ We know deterministically what would be the next state.

→ Only one next state.

→ No choice.

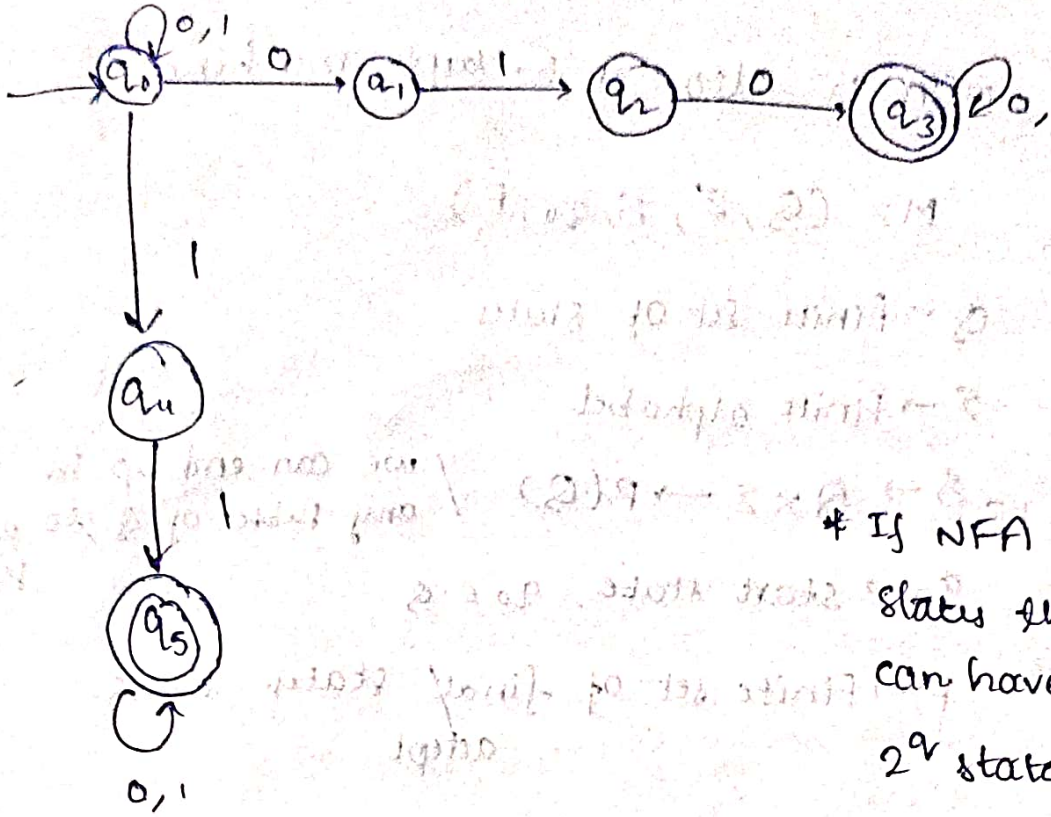
→ In NFA, several choices may exist for the next state at any point.

$\delta(q_i, a)$ ↔ ϕ → no state

q_j → goes to 1 state

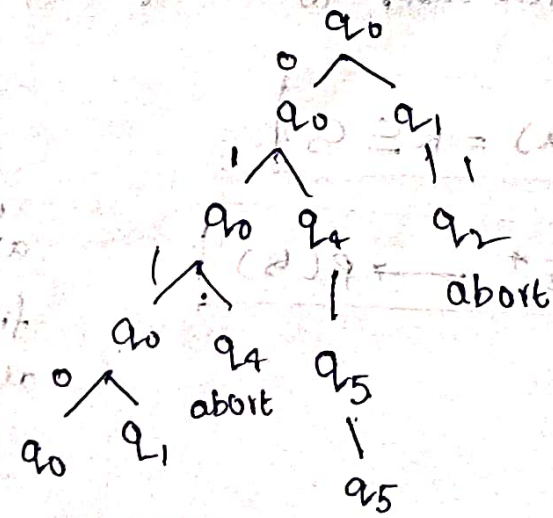
$R \subseteq Q$ → goes to subset of set of states

Eg: $\Sigma = \{0,1\}$ / Language that has substrings 010, 11.



* If NFA has q states then DFA can have max 2^q states.

Eg: Computation 0 1 1 0

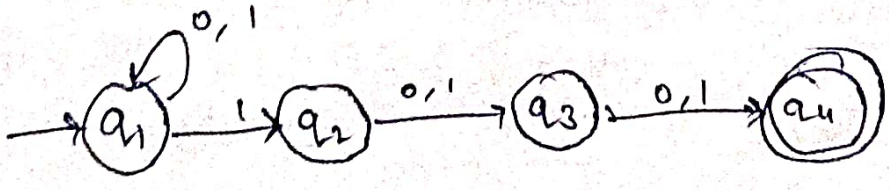


Time complexity for DFA → $O(2^n)$

∴ DFA easier for programming.

for L_{11} (DFA is very complex)

NFA:



→ Constructing NFA is sometimes easier than constructing DFA.

→ Every NFA has equivalent DFA.

* Formally, NFA is also a 5-tuple machine.

$$M = (Q, \Sigma, \delta, q_0, F)$$

$Q \rightarrow$ Finite set of states

$\Sigma \rightarrow$ Finite alphabet

transition fn $\leftarrow \delta \rightarrow Q \times \Sigma \longrightarrow P(Q)$ / we can end up in any subset of Q, so power set

$q_0 \rightarrow$ start state, $q_0 \in Q$

$F \rightarrow$ Finite set of final states $F \subseteq Q$.
accept

* $L(M)$: The language accepted by NFA M is.

$$L(M) = \{ x \in L \mid x \text{ can take the machine } M \text{ from its initial state to one of its final states} \}$$

$$\boxed{\hat{\delta}(q_0, x) = R \subseteq Q}$$

$$* \quad \boxed{\hat{\delta} : Q \times \Sigma^* \longrightarrow P(Q)}$$

Σ^* → any string from input alphabets Σ.

* Formally

$$\boxed{L(M) = \{ x \in L \mid \hat{\delta}(q_0, x) \cap F \neq \phi \}} *$$

→ If L is accepted by DFA then it is also accepted by NFA because every NFA has an equivalent DFA.

→ Is there a language that is accepted by NFA but not by DFA ? NO

$L(M) = M \xrightarrow{\text{NFA}} M' \xrightarrow{\text{DFA}}$

• Equivalence of NFA & DFA:

* **Theorem**: Every NFA has an equivalent DFA.

Proof:

Proof by construction: Build an equivalent DFA for the given NFA.

Let $M = (Q, \Sigma, \delta, q_0, F)$ be the NFA recognizing some language $L(M)$.

We construct a DFA $M' = (Q', \Sigma, \delta', q_0', F')$

→ $Q' = P(Q)$

→ $\delta'(R, a) = R'$, $R \subseteq Q'$ and $a \in \Sigma$

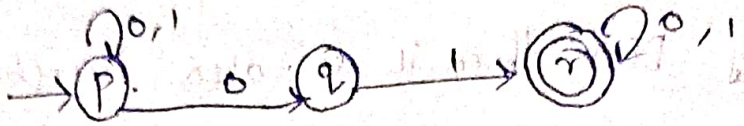
$$R' = \bigcup_{r \in R} \delta(r, a)$$

→ $q_0' = \{q_0\}$

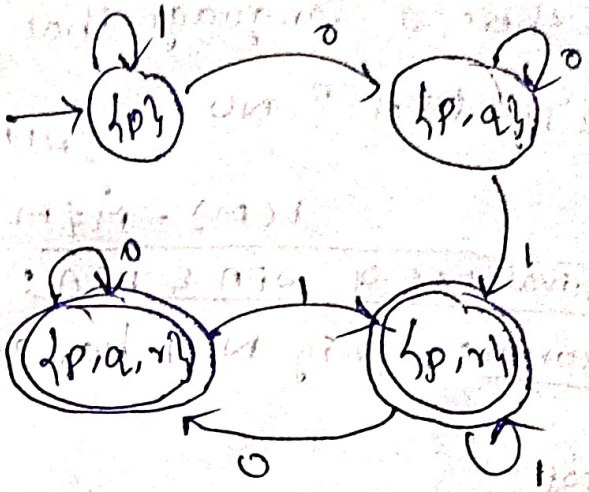
→ $F' = \{R \in Q' \mid R \text{ contains an accept/final state of } M\}$

$= \{R \subseteq Q \mid R \cap F \neq \phi\}$

Eg:



	0	1
→ {p}	{p,q}	{p}
{p,q}	{p,q}	{p,r}
* {p,r}	{p,q,r}	{p,r}
*{p,q,r}	{p,q,r}	{p,r}

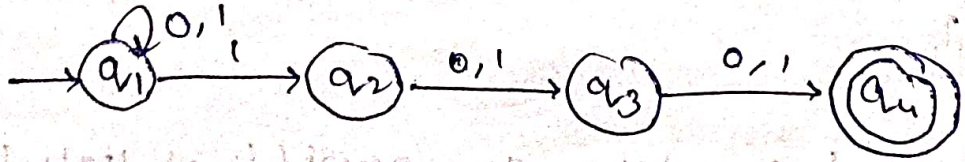


** → no new states
so stop

* → final states
because they
contain final
state of NFA

Eg: $L_{11} = \{x \in \{0,1\}^* \mid$ the third bit from right end is 1 in $x\}$.

NFA



	0	1
→ $\{q_1\}$	$\{q_1\}$	$\{q_1, q_2\}$
$\{q_1, q_2\}$	$\{q_1, q_3\}$	$\{q_1, q_2, q_3\}$
$\{q_1, q_3\}$	$\{q_1, q_4\}$	$\{q_1, q_2, q_4\}$.
$\{q_1, q_2, q_3\}$	$\{q_1, q_3, q_4\}$	$\{q_1, q_2, q_3, q_4\}$.
* $\{q_1, q_4\}$	$\{q_1\}$	$\{q_1, q_2\}$
* $\{q_1, q_2, q_4\}$	$\{q_1, q_3\}$	$\{q_1, q_2, q_3\}$.
* $\{q_1, q_3, q_4\}$	$\{q_1, q_4\}$	$\{q_1, q_2, q_4\}$
* $\{q_1, q_2, q_3, q_4\}$	$\{q_1, q_3, q_4\}$	$\{q_1, q_2, q_3, q_4\}$

* NFA with ε-transitions:

→ Generalization of NFA.

→ It allows transitions on empty strings.

* What language is accepted by NFA with ε-transitions?

Ans: Regular languages.

~~* ε-closure:~~

* ε-closure:

if $P_i \in Q$, then ε-closure(P_i) = $\{q_j \mid q_j$ is reachable from P_i using 0 or more ε-transitions$\}$

Formally

$$\varepsilon\text{-closure}(P_i) \triangleq \bigcup_{q_i \in P} \varepsilon\text{-closure}(q_i)$$

$M = (Q, \Sigma, \delta, q_0, F)$

$\delta : Q \times \Sigma \cup \{\varepsilon\} \longrightarrow P(Q)$

$\hat{\delta} : Q \times \Sigma^* \longrightarrow P(Q)$

$\hat{\delta}(p, x) = R \subseteq Q \rightarrow$ The set of all states M can be in starting from p on input x.

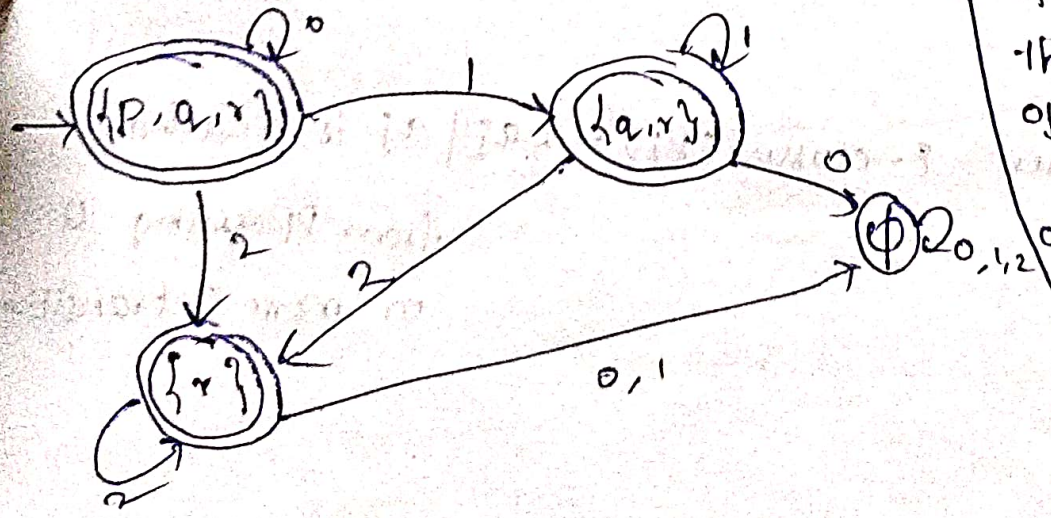
Eg:

Step 1: Find ε closure for all states

ε-closure(p) = {p, q, r}

ε-closure(q) = {q, r}

ε-closure(r) = {r}

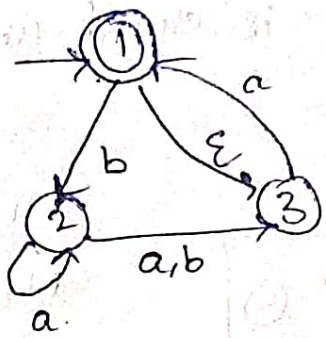


* while checking single alphabet if Eg: p on 0 ≠ ∅ then add ε-closure of p.

p

0 {p, q, r}

Eg: To Convert N_4 NFA to DFA



	1	2	3
a	ϕ	2,3,2,3	1,1,3
b	2,2	3,3	ϕ

ε-closure of all state

ε-closure(1) = {1,3}

ε-closure(2) = {2}

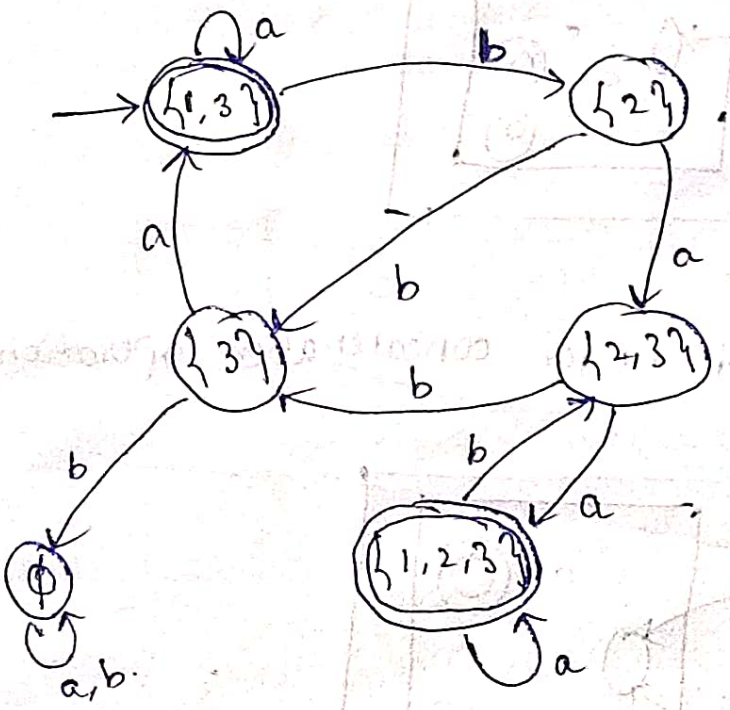
ε-closure(3) = {3}

	2
a	2,3,2,3 (ε-clos)
b	3,3

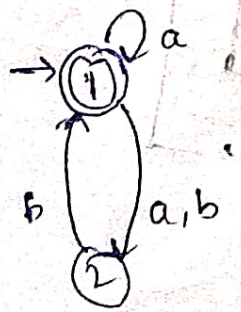
	1	3
a	ϕ	1,1,3
b	2,2	ϕ

	2	3
a	2,3,2,3	1,3
b	3,3	ϕ

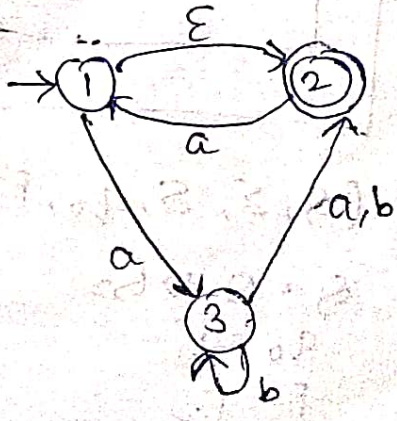
	3
a	1,1,3
b	ϕ



Eg: N_5



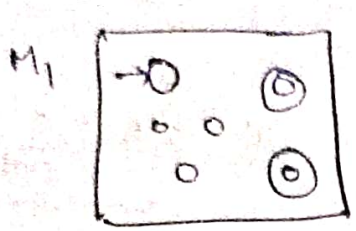
N_6

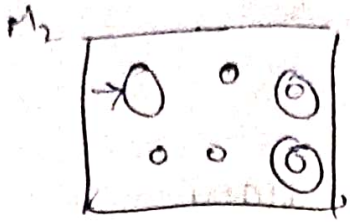


* Properties of Regular language (continuation of operations on R.L)

→ The class of Regular language is closed under union operation

M_1 & M_2

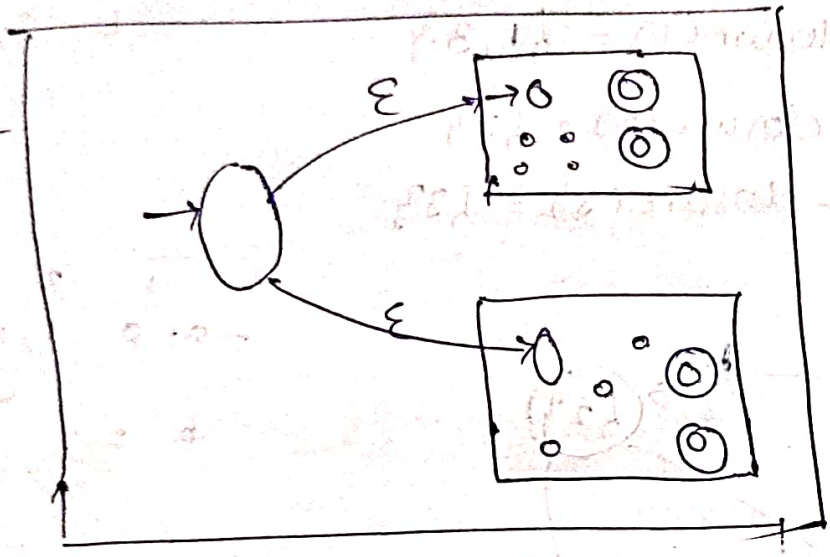




$M_1(Q_1, \Sigma, \delta_1, q_1, F_1)$

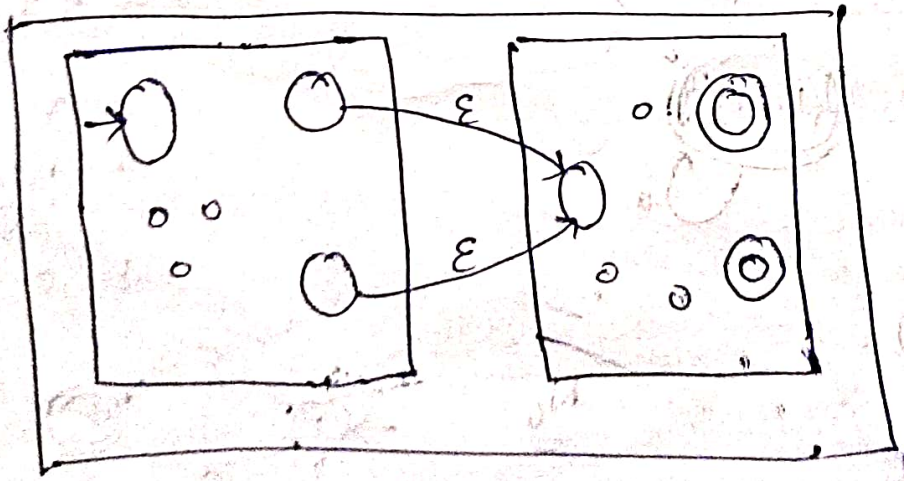
$M_2(Q_2, \Sigma, \delta_2, q_2, F_2)$

$M_1 \cup M_2$



→ The class of R.L is closed under concatenation operations

$M_1 \cdot M_2$



$(Q, \Sigma, \delta, q_0, F)$

$Q = Q_1 \cup Q_2$

$q_0 = q_1$

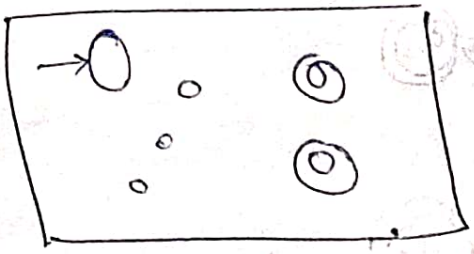
$F = F_2$

$$\delta(q,a) = \begin{cases} \delta_1(q,a) & q \in Q_1 \\ & q \notin F_1 \\ \delta_1(q,a) & q \in F_1 \\ & a \neq \varepsilon \\ \delta_1(q,a) \cup \{q_2\} & q \in F_1 \\ & a = \varepsilon \\ \delta_2(q,a) & q \in Q_2 \end{cases}$$

→ The class of R.L is closed under star operation.

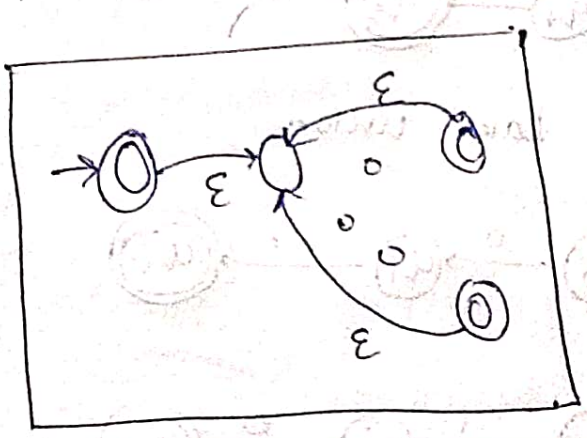
↳ kleen closure.

M_1



$M_1 (Q_1, \Sigma, \delta_1, q_1, F_1)$

M



$M = (Q, \Sigma, \delta, q_0, F)$

$Q = Q_1 \cup \{q_0\}$

$q_0 = \{q_0\}$

$F = F_1 \cup \{q_0\}$.

$$\delta(q,a) = \begin{cases} \delta_1(q,a) & q \in Q \text{ \& } q \notin F_1 \\ \delta_1(q,a) & q \in F_1 \text{ \& } a \neq \varepsilon \\ \delta_1(q,a) \cup \{q_1\} & q \in F \text{ \& } a = \varepsilon \\ \{q_1\} & q = q_0 \text{ \& } a = \varepsilon \\ \phi & q = q_0 \text{ \& } a \neq \varepsilon \end{cases}$$

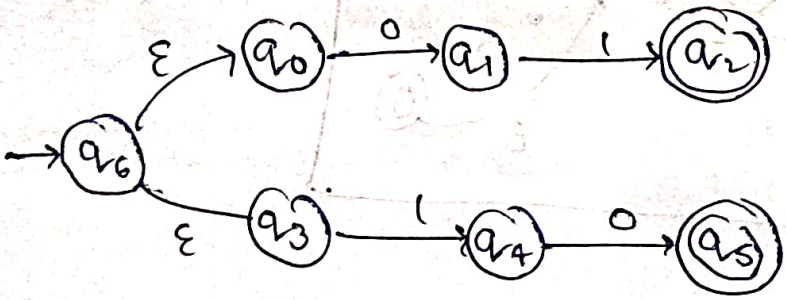
6) Design a DFA to accept Σ^* where $\Sigma = \{01, 10\}$

Step-1 : M_1

M_2

Step-2 :

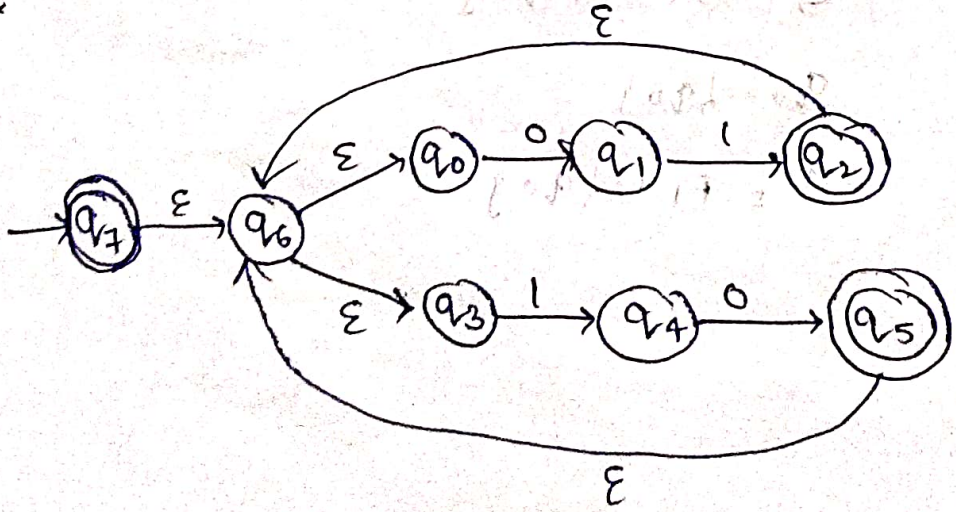
Since 01 or 10 take union.



} for ε

Step-3 :

for Σ^*



NFA

Step-4 : Conversion to DFA

Find ε-closure.

ε-closure(q_7) = $\{q_7, q_6, q_0, q_3\}$

ε-closure(q_6) = $\{q_6, q_0, q_3\}$

ε-closure(q_0) = $\{q_0\}$

ε-closure(q_1) = $\{q_1\}$

ε-closure(q_2) = $\{q_2, q_6, q_0, q_3\}$

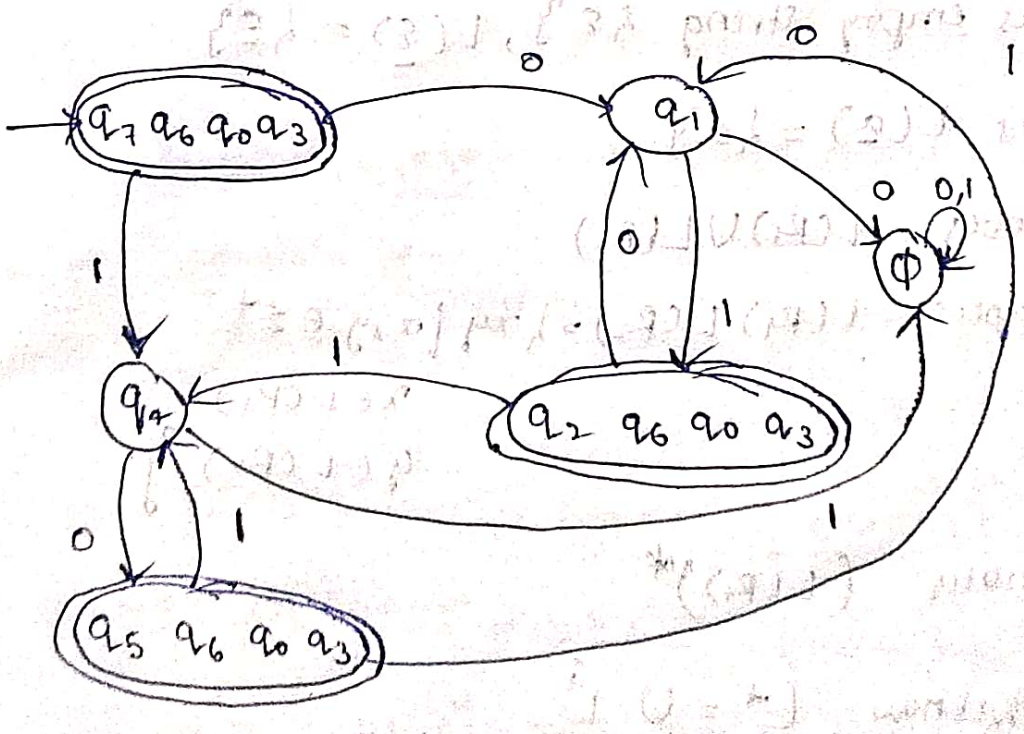
ε-closure(q_3) = $\{q_3\}$

ε-closure(q_4) = $\{q_4\}$

ε-closure(q_5) = $\{q_5, q_6, q_0, q_3\}$

Step-5 : Draw DFA

Since $q_7 \xrightarrow{\phi} q_2, q_6, q_0, q_3$.



	q_7 q_6 q_0 q_3
0	ϕ ϕ q_1 ϕ
1	ϕ ϕ ϕ q_4

	q_1
0	ϕ
1	q_2 q_6 q_0 q_3

	q_4
0	q_5 q_6 q_0 q_3
1	ϕ

	q_2 q_6 q_0 q_3
0	ϕ ϕ q_1 ϕ
1	ϕ ϕ ϕ q_4

	q_5 q_6 q_0 q_3
0	ϕ ϕ q_1 ϕ
1	ϕ ϕ ϕ q_4

* Regular Expression (RE):

→ A RE R over an alphabet Σ is defined as follows

1) $\underline{\phi}$, $\underline{\varepsilon}$, $\underline{a}$ are REs over Σ.

→ 2) If R_1 & R_2 are REs then

(R_1)

$R_1+R_2 \mid R_1 \cup R_2$

$R_1 R_2 \mid R_1 \cdot R_2$

$(R_1)^*$

are also REs

→ Every RE over Σ denotes a language over Σ.

→ If R is a RE, then L(R) is a language denoted.

$\underline{\phi}$ - denotes empty language ϕ, $L(\underline{\phi}) \subseteq \Sigma^*$

$\underline{\varepsilon}$ - denotes empty string $\{\varepsilon\}$, $L(\underline{\varepsilon}) = \{\varepsilon\}$

$\underline{a}$ - denotes $L(\underline{a}) = \{a\}$.

R_1+R_2 denotes $L(R_1) \cup L(R_2)$

$R_1 R_2$ denotes $L(R_1)\,L(R_2) = \{xy \mid x, y \in \Sigma^+$, $x \in L(R_1)$, $y \in L(R_2)\}$

$(R_1)^*$ denotes $(L(R_1))^*$

// We know $L^+ = \bigcup_{i \geq 0} L^i$

Eg: $\Sigma = \{0,1\}$

$\underline{0}$ denotes $L(\underline{0}) = \{0\}$

$\underline{1}$ denotes $L(\underline{1}) = \{1\}$

$\underline{0} + \underline{1}$ denotes $L(\underline{0}) \cup L(\underline{1}) = \{0,1\}$

* $(\underline{0} + \underline{1})^*$ denotes. $\{0,1\}^* = \Sigma^*$

→ Several REs might denote the same language.

Eg: $\{\underline{0} + \underline{1}\}^* = (\underline{0}^* \underline{1}^*)^*$

* RE

Operands - RE

Operators

Precedence

→ Brackets

→ Union

→ Concatenation

→ Star | Kleen closure.

1 > 4 > 3 > 2

* $((\underline{0})^* (\underline{1})^*)^* = (0^* 1^*)^*$

* Identify the languages of the following RE

$\Sigma = \{0,1\}$

$\underline{0}^* \underline{1} \underline{0}^* = \{w \mid w$ has exactly one $1\}$

$(\underline{0} + \underline{1})^* \underline{1} (\underline{0} + \underline{1})^* = \Sigma^* \underline{1} \Sigma^* = \{w \mid w$ has atleast one $1\}$

$(\underline{0} + \underline{1})^* \underline{0} \underline{0} \underline{1} (\underline{0} + \underline{1})^* = \Sigma^* \underline{0} \underline{0} \underline{1} \Sigma^* = \{w \mid w$ has substring $001\}$

$((0+1)(0+1))^* = (\Sigma\Sigma)^*$ = {w | w has even number of symbols}

$(\Sigma\Sigma)^0 = \varepsilon$

$(\Sigma\Sigma)^1$ = length 2

↙ 1 from this, 1 from this ↘

$(\Sigma\Sigma)^2$ = length 4

⋮

$(\Sigma\Sigma)^n$ = length 2n

$((0+1)(0+1)(0+1))^* = (\Sigma\Sigma\Sigma)^*$ = {w | w ~~has length~~ contain strings of length multiple

→ $0(0+1)^*0 + 1(0+1)^*1 + 0 + 1$

= $0\Sigma^*0 + 1\Sigma^*1 + 0 + 1$. = {w | w has strings that has same start and end symbols}

→ $1^*(01^+)^*$ = {w | w has strings such that every 0 in w is followed by atleast one 1 and equal number of 1's}

↳ without ε

$(01^+)^0 = \varepsilon$

$(01^+)^1 = (01^1)^1$,

$(01^+)^2 = (01^1)^2 = 01$

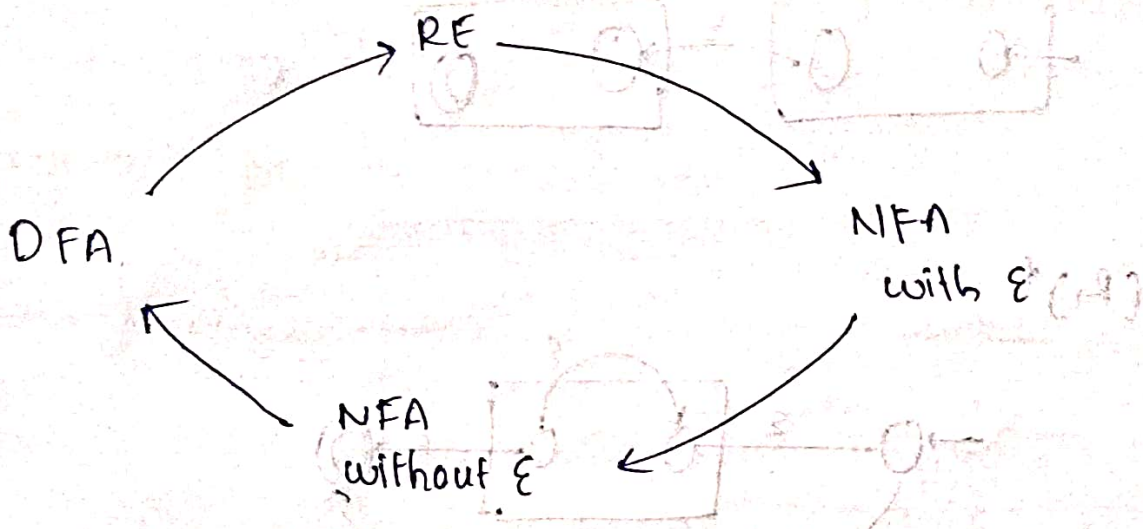
$= (01^2)^2 = 0112$

- $\Sigma = \{a, b, c\}$

$(a+b)^* (c(a+b)^* c (a+b^*))^*$

→ Even number of c's

→ $c^*(a+bc^*)^* = \{w \mid w$ contains strings in which ac does not occur as a substring$\}$



* For equivalence $L(RE) \subseteq L(RL)$ & $L(RL) \subseteq L(CRE)$

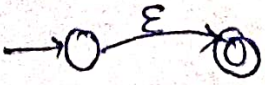
1) Given any RE R, L(R) is a regular language.

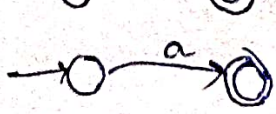
2) Given any RL L, there is a ~~regular~~ RE R. such that $L(R) = L$.

1) Proof by Induction

i) Basis

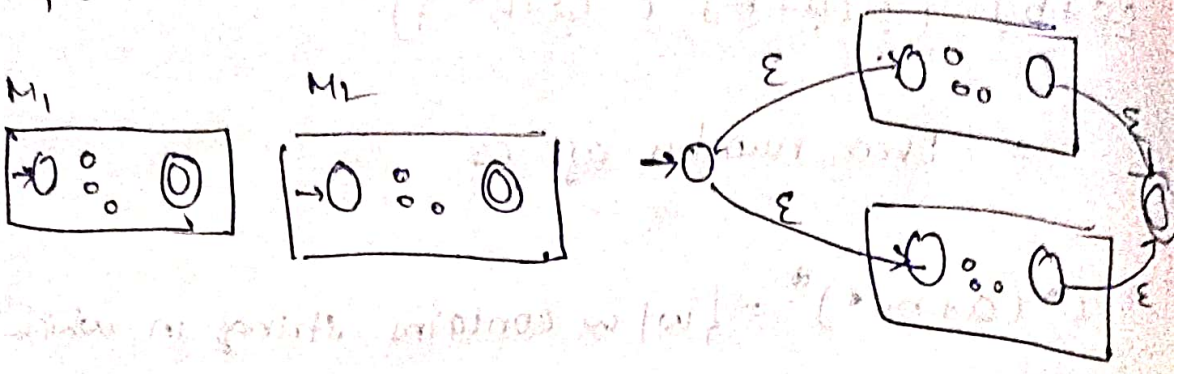
ϕ

ε 

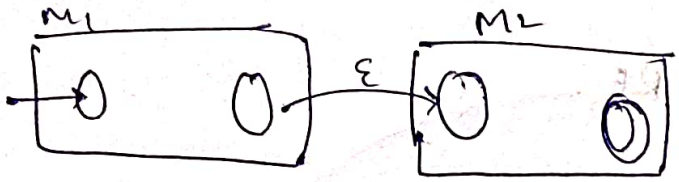
a 

2) Induction

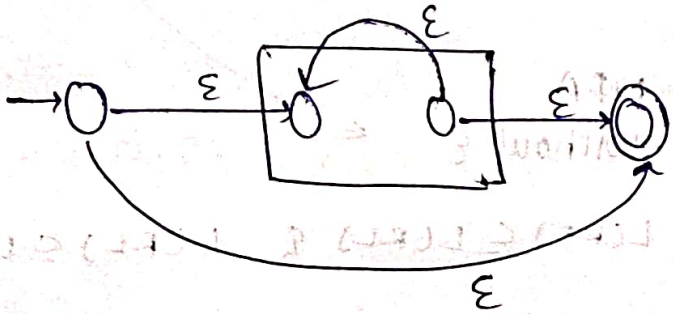
$R_1 \cup R_2$



$R_1 R_2$



$(R_1)^*$



For all cases inductively we have proved given any RE R, L(R) is a regular language.

//

2) Given any RL L, there is a regular Expression R, such that L(R) = L using GNFA
↳ generalized NFA

Three rules for GNFA

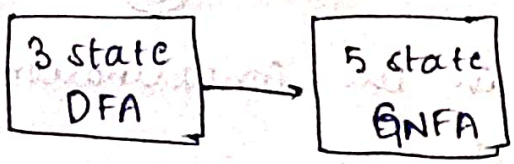
i) The start state has transition arrows going to every other state but no arrows coming in from any other state.

ii) There is only a single accept state, and it has arrows coming in from every other state but no arrows going to any other state. Furthermore, the accept state is not the same as the start state.

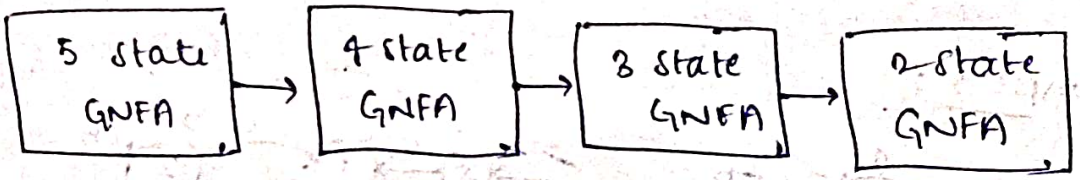
iii) Except for the start ~~sta~~ and accept states, one arrow goes from every state to every other state and also from each state to itself.

For example : * DFA to RE conversion *

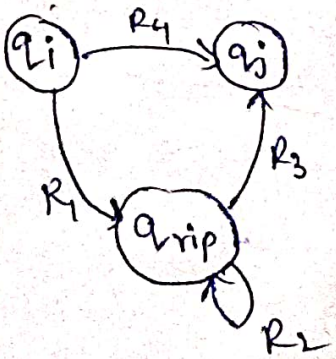
Step-1 : Adding 2 states

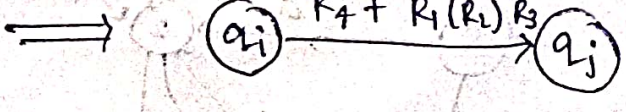


Step-2 : Removing until left with added ones



Example :



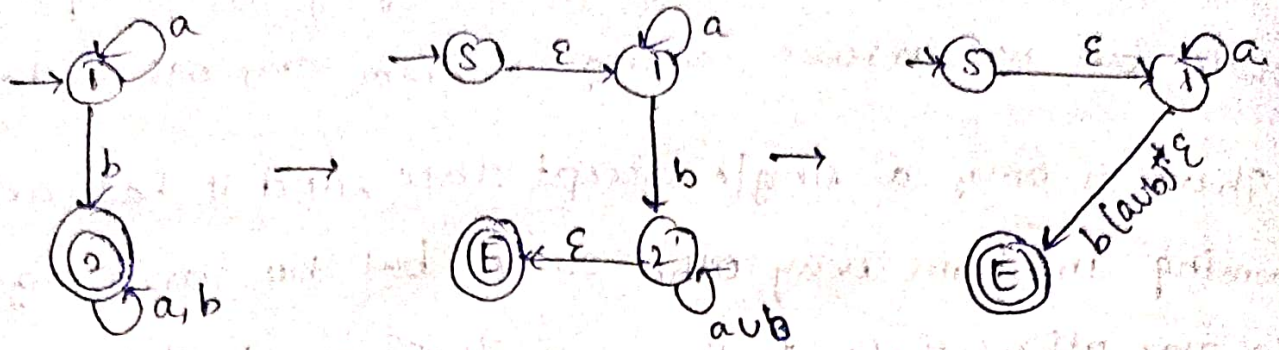


all possible paths to reach q_j

R_4 ✓

$R_1(R_2)^*R_3$ ✓

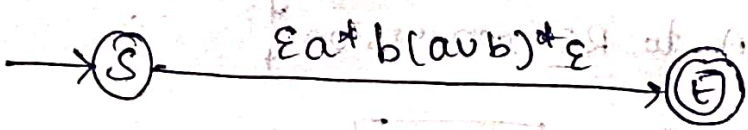
Example 2 :



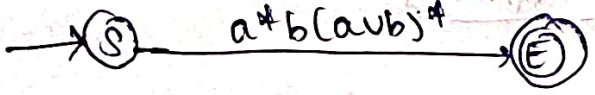
2 → victim

So bypass from 1 to E

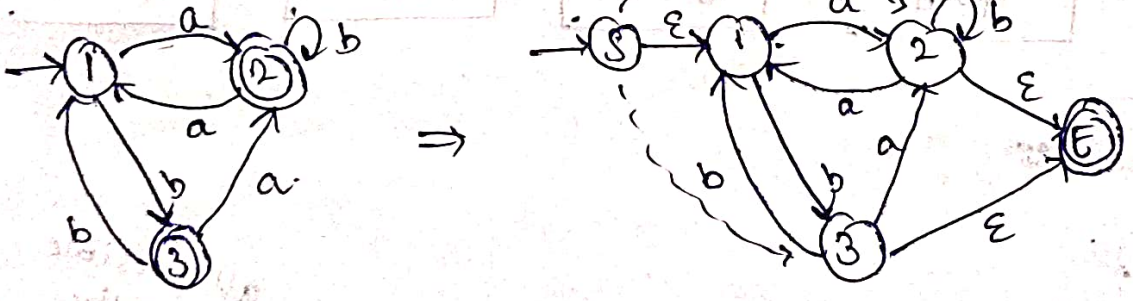
1 → victim

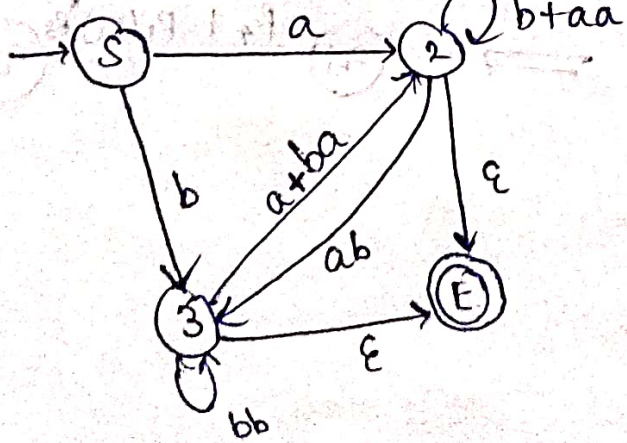


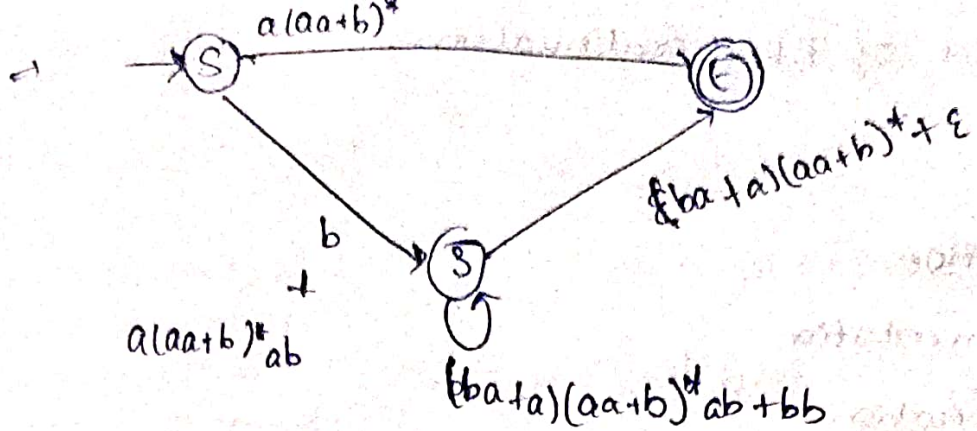
Eliminate ε only in the concatination operation

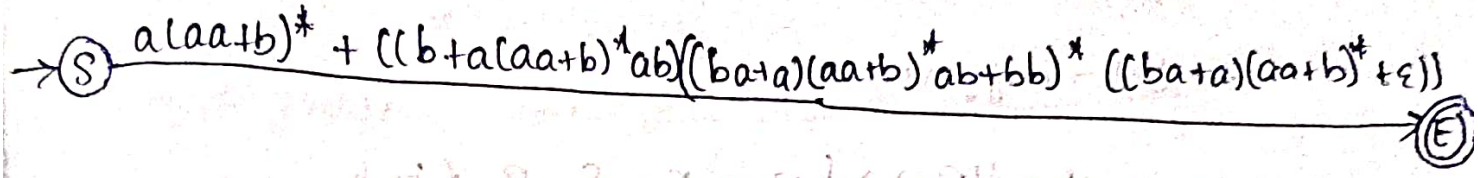


Example 3 :









* Algebraic laws for regular expression:

$L+M = M+L$ ⟶ commutative.

$L+(M+N) = (L+M)+N$ ⟶ Associative.

$L(MN) = (LM)N$.

$\phi + L = L + \phi = L$

$\varepsilon L = L\varepsilon = L$ } Identity

$\phi L = L\phi = \phi$ ⟶ Annihilator

$L(M+N) = LM + LN$

$(M+N)L = ML + NL$ } distributive.

$L+L = L$

$(L^*)^* = L^*$ } Idempotent

$\phi^* = \varepsilon$

$\varepsilon^* = \varepsilon$

* Closure properties of RL (continuation)

→ Union
→ Intersection
→ Complementation
→ Concatenation
→ Kleen closure
→ Set-difference.

$$F = \{(P_1, P_2) \mid P_1 \in F_1 \ \& \ P_2 \notin F_2\}$$

$$= L_1 \cap \overline{L_2}$$

→ Reversal.

* PDF -5-a

1.20

a) $a^*b^* = \{\varepsilon, b, bb, a, ab, abb\}$ ✓

$= \{ba, aba, \ldots\}$ ✗

b) $(a \cup ba \cup bb)\Sigma^* \Rightarrow (a+ba+bb)(a+b)^*$

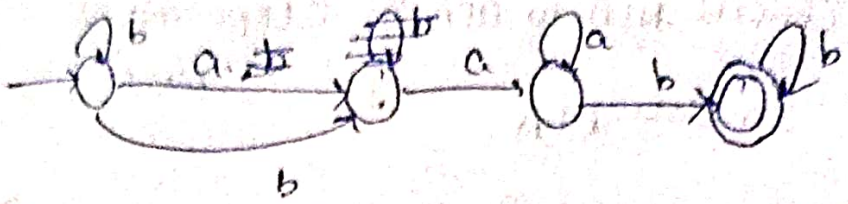
$= \{a, aa, ab, aaa, aab, aba, abb\}$ ✓

$= \{\varepsilon, b\}$ ✗

1.28

c) $(a \cup b^+)a^+b^+$

$= (a + b^+)a^+b^+$



* Decision problems concerning R.L:

→ Decision problems are those computational problems whose output is yes/no.

→ Given an input RL L & w is a string, is $w \in L$? also called as language membership problem.

1) L is represented by DFA M, $L = L(M)$

Input : M, w — M → DFA, w → string

Is $w \in M$?

Simulate $\hat{\delta}(q_0, w)$ → If this simulation ends in some q_f the answer is YES. else NO.

Time complexity : $O(n)$, $n \rightarrow$ length of w.

2) L is represented by NFA M., L = L(M)

Input : M, w , M → NFA

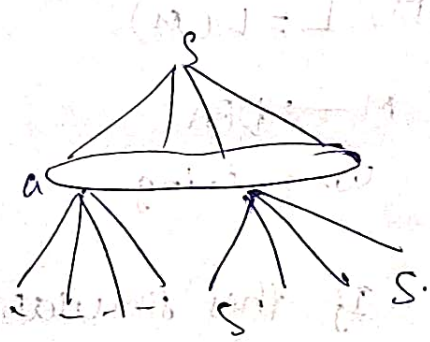
Is w ∈ M?

Option-1 : S-1 : Convert NFA to DFA } → Exponential time

S-2 : Simulate DFA

Option-2 : Keep track of the set of states M can be in as we scan w symbol by symbol

To process one symbol, the worst case is S^2 where ~~Q = {s1}~~ . |Q| = S , to process a string we need $O(nS^2)$ where |w| = n.



3) L is represented by NFA with ε transitions., L = L(M)

Input M, w, M is NFA with ε

Is w ∈ L(M)?

Step-1 : Find ε-closures. — $O(S^3)$

Same Step-2 & Step-3

Combining all the steps we get $O(ns^2 + s^3)$

4) L is represented by RE R

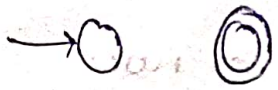
Step-1: Convert RE, R to NFA M. — polynomial

Step-2: Decide if $w \in L(M)$

Same as NFA which is also polynomial.

* Some decision problems

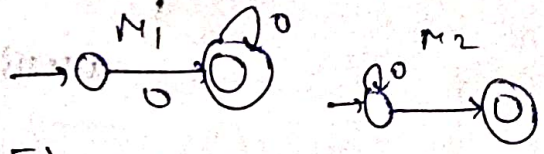
i) Given L a regular language, is L empty.?



ii) Given two DFAs M_1 & M_2, is $L(M_1) = L(M_2)$.

* Minimization of DFA:

→ For every language L, there exists a unique (up to re-labling of the states) minimal DFA M, such that $L = L(M)$.

→ NOT true for NFA. Eg: 

→ Given that $M = (Q, \Sigma, \delta, q_0, F)$

$\hat{\delta} : Q \times \Sigma^* \to Q$

$\hat{\delta}(q, \varepsilon) \to q$

$\hat{\delta}(q, a) = \delta(q, a)$ } if only 1 symbol is present

$\hat{\delta}(q, w_1 \cdots w_{k+1}) = \delta(\hat{\delta}(q, w_1 \cdots w_k), w_k))$

* Distinguishable :

→ A string $w \in \Sigma^*$ distinguishes state q_1 from q_2 if $\hat{\delta}(q_1, w) \in F \iff \hat{\delta}(q_2, w) \notin F$.

→ ε distinguishes final/accept states from non-final/non-accept states.

* Myhill-Nerode theorem:

- Minimization of DFA
- Equivalence of DFA
- To prove a language is regular or not

→ Minimization of DFA

Step-1 : Remove non-reachable/dead states
↳ has only outgoing transitions, no incoming transitions.

Step-2 : Apply table filling (M-N theorem) algorithm.

To get $E_m = \{[q]\}$ q is an accessible state of M

E_m → set of distinguishable states

$M_{min} = (Q_{min}, \Sigma, \delta_{min}, q0_{min}, F_{min})$

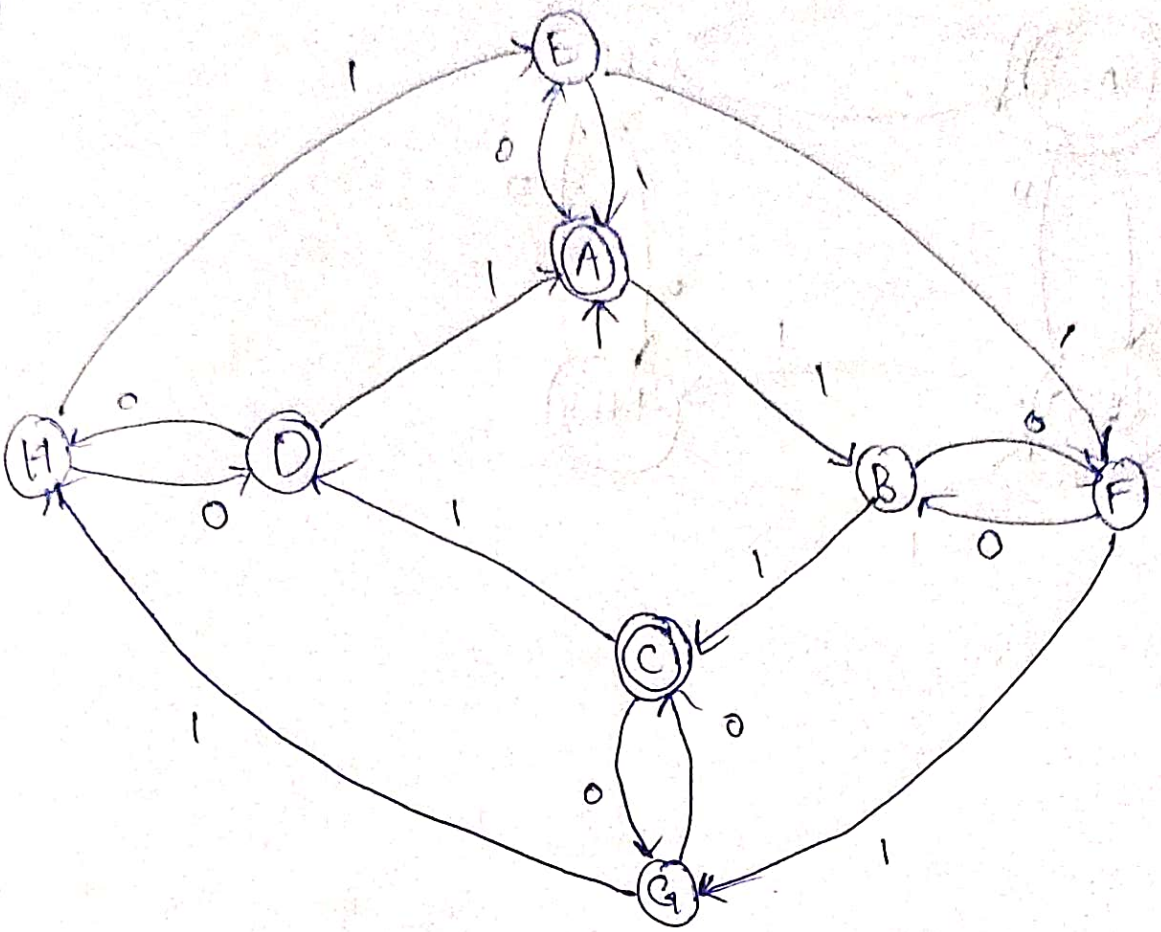
$Q_{min} = E_m$

$q0_{min} = [q0]$

$F_{min} = \{[q] \mid q \in F\}$

$\delta_{min}([q], a) = [\delta(q, a)]$

Eg:



Step-1 : No dead states

Step-2 : Find equivalence states (empty ones).

by eliminating distinguishable states

x → distinguishable

	A	B	C	D	E	F	G
B	x						
C		x					
D	x		x				
E	x	x	x	x			
F	x	x	x	x	x		
G	x	x	x	x		x	
H	x	x	x	x	x		x

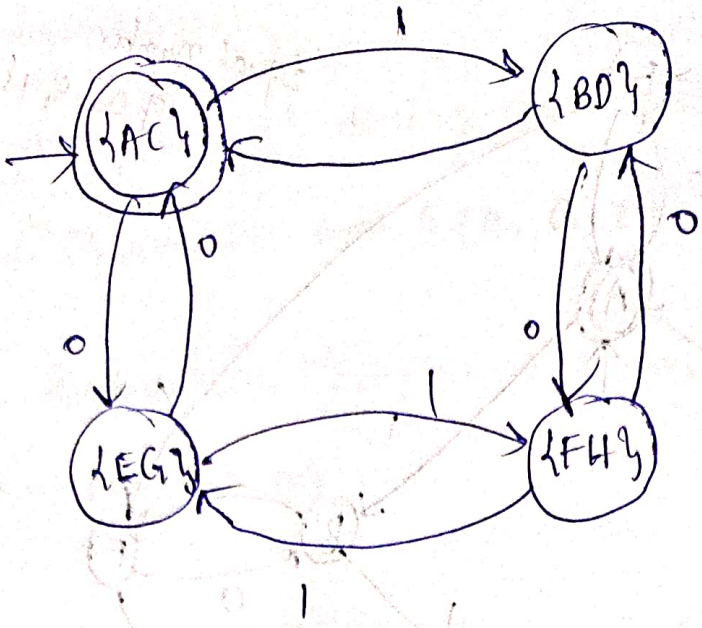
empty boxes → equivalence states

F: 0 → B, 1 → G
G: 0 → C

One NF
One F
So distinguishable

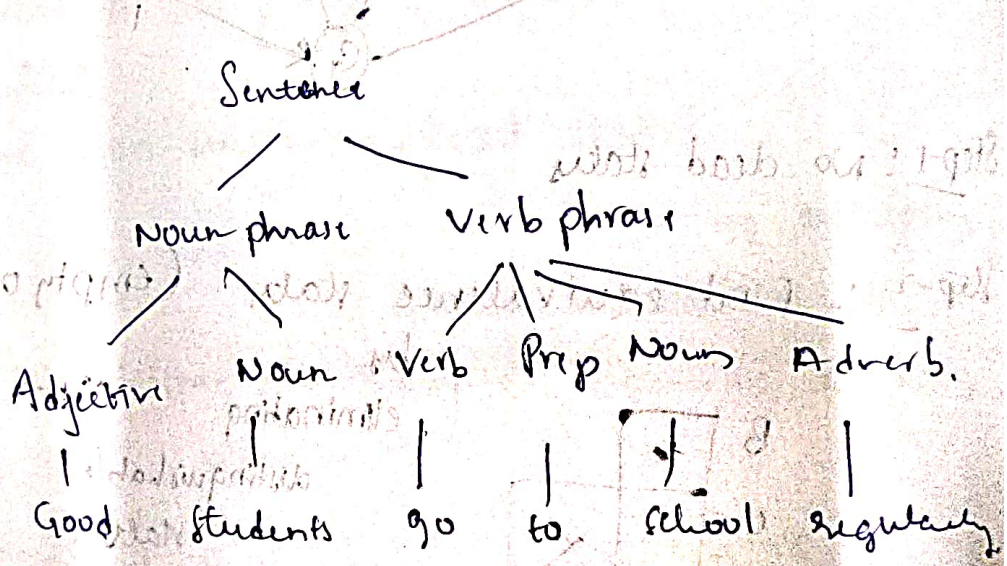
G: 0 → C, 1 → H
H: D

→ don't write last state
↑ don't write first state



* Grammar

Good students go to school regularly.



<sentence> → <nounphrase> <verbphrase>

<nounphrase> → <Adj> <nouns>

<Adj> → good

<nouns> → students | school

<verbphrase> → <verb> <prep> <nouns> <Adverb>

<verb> → go

<prep> → to

<adverb> → regularly.

Eg: $\Sigma = \{0,1\}^*$

G_1: $S \to 0S1$

$S \to \varepsilon$

S

0 S 1

00 S 11

000 S 111

000 111

$\{0^n 1^n \mid n \geq 0\} \subseteq \{0,1\}^*$

* A grammar G is a tuple. $G = (V, \Sigma, P, S)$ (some books Σ as T)

V: finite set of non terminals

will be replaced by a non-terminal or a terminal also does not appear in the end string of a derivation

$V = \{S\}$

T/Σ: finite set of terminals $\Sigma = \{0,1\}$

P: finite set of production / rewriting rules. $P = \{S \to 0S1, S \to \varepsilon\}$

S: ~~Start~~ Unique start symbol. $S = S$

* $\to V \cap \Sigma = \emptyset$

$\to S \in V$

$\to P \subseteq V \times (V \cup \Sigma)^*$

$\subseteq (V \cup \Sigma)^*$

Eg: G_2 : $V = \{S\}$

$\Sigma = \{a, b\}$

P : $\{S \to aSa, S \to bSb, S \to \varepsilon\}$

$S = S$

we can replace S with any of P

asa

absba

abasaba

abaεaba

abaaba

∴ Even length palindromes

Formally :

We have a string $\alpha_1 X \alpha_2$, such that $\alpha_1, \alpha_2 \in (V \cup \Sigma)^*$

if $X \to \gamma \in P$, then $\alpha_1 X \alpha_2 \Rightarrow \alpha_1 \gamma \alpha_2$

(derives)

if $\gamma_1 \Rightarrow \gamma_2 \Rightarrow \gamma_3 \cdots \Rightarrow \gamma_n$, γ. by using ^transitive property $\gamma_1 \overset{*}{\Rightarrow} \gamma_n$ (length not n)

(reflexive)

$\gamma \overset{*}{\Rightarrow} \gamma$

* The language $L(G) \subseteq \Sigma^*$ is associated with a grammar $G = (V, \Sigma, P, S)$ is the set:

$$L(G) = \{ x \in \Sigma^* \mid S \overset{*}{\Rightarrow} x \}$$

$L(G)$ is the language generated by grammar G.

→ Context free grammar (CFG):

* A grammar $G = (V, \Sigma, P, S)$ is a context free grammar, if each element of production rules P is of the form $X \to \alpha$

where $X \in V$ & $\alpha \in (V \cup \Sigma)^*$

→ A language L is a context free language, if there is a context free grammar G such that $L = L(G)$

→ $L \subseteq L(G)$

$L(G) \subseteq L$

→ Daily usage in Arithmetic operations.

Eg: $G_1 = \{\{S\}, \{0,1\}, \{S \to 0S1, S \to \varepsilon\}, S\}$

$L(G_1) = \{0^n 1^n, n \geq 0\}$

Case1: $L(G_1) \subseteq \{0^n 1^n \mid n \geq 0\}$

Case1: $L(G_1) \geq \{0^n 1^n, n \geq 0\}$

Case1: → always about length

Basis

$n=0$ ε $S \to \varepsilon$ ✓

$n=1$ 01 $S \Rightarrow 0S1 \Rightarrow 01$ ✓

Induction hypothesis:

→ for $n=k$, assume it is true.

Induction: prove for $n = k+1$ $0^{k+1}1^{k+1}$

$$S \overset{*}{\Rightarrow} \underbrace{00^k 1^k}_{\alpha} 1 \Rightarrow 0\alpha 1$$

We already $0^k 1^k$ (α) can be derived from S

Therefore by induction, hence proved.

Case-2: → always about no. of steps

Basis: In 1 step, the terminal string that can be derived from S are $S \Rightarrow \varepsilon \in \{0^n 1^n \mid n \geq 0\}$.

In 2 steps

$S \Rightarrow 0S1 \Rightarrow 01 \in \{0^n 1^n \mid n > 0\}$

Hypothesis:

lets assume true for k steps

Induction:

→ Prove for k+1 steps

$$S \to \alpha_1 \to \alpha_2 \to \alpha_3 \cdots \alpha_{k+1}$$

$$\to bSb \to \alpha_2 \to \alpha_3 \cdots \alpha_{k+1}$$

(true)

Eg: $G_4 = \{\{S\}, \{a,b\}, \{S \to aSa,\ S \to bSb,\ S \to \varepsilon,\ S \to a,\ S \to b\}, \{S\}\}$

$L(G_4) = \{w = w^R \mid w \in \{a,b\}^*\}$

Case-1: $\{w = w^R \mid w \in \{a,b\}^*\} \subseteq L(G_4)$

Case-2: $L(G_4) \subseteq \{w = w^R \mid w \in \{a,b\}^*\}$

Case-1:

Basis:

If $|w| = 0$ or $|w| = 1$.

ε, a, b by using Production rule 3,4,5

we can prove basis.

Hypothesis

let for $|w| \leq k$ where $(w = w^R)$, $S \overset{*}{\Rightarrow} w$ is true

Induction :

we need to ~~start~~ show $|w| = k+1$, $S \overset{*}{\Rightarrow} w$ is true.

$w = axa$ or bxb where $|x| < k$

$S \Rightarrow asa \overset{*}{\Rightarrow} axa$ ($\because |x| < k$, $S \overset{*}{\Rightarrow} w$ is true).

Adding same start symbol & end symbol to x will also make the resultant string as palindrome.

→ Similarly for bxb.

Case-2 :

Basis : in one step

$S \to \varepsilon$

$S \to a$

$S \to b$

$\varepsilon, a, b \in$ set of palindromes.

in two steps :

$S \to asa \to aa$

$\to aba$ $\in$ set of palindromes

$\to aaa$

Hypothesis :

Assume that it is true for k steps

Induction:

$S \Rightarrow \alpha_1$

$\Rightarrow \alpha_2$

$\Rightarrow \alpha_3$

$\Rightarrow \alpha_{k+1}$

$\alpha_1 \Rightarrow \underbrace{asa}_{\alpha_2}$

$\Rightarrow a\alpha_2 a$

* (Stack overflow)

* Conventions

→ Non terminals with capital letters

→ Terminals with small letters & numbers

→ Terminal strings with small letters from the end of the alphabet (w, x, y, z)

→ Strings from $(V \cup \Sigma)^*$ with greek letters.

Intermediate

* Every regular language is a context-free language —

DFA, M ↓ ↑ CFG, G

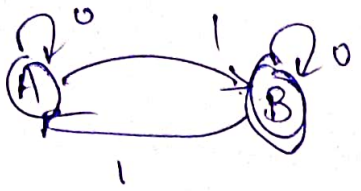
$L(M) \supset L(G)$

$L(M) \subseteq L(G)$

$L(G) \subseteq L(M)$

DFA to CFG

Eg:



Odd number of 1's:

$A \rightarrow 0A$

$A \rightarrow 1B$

$B \rightarrow 0B$

$B \rightarrow 1A$

$A \rightarrow 1$

$B \rightarrow 0$

$x \rightarrow \alpha$

$x \in V$

$\alpha \in (V \cup \Sigma)^*$

→ Proof by construction:

Let L be a RL. Then $M = (Q, \Sigma, \delta, q_0, F)$ is a DFA that accepts L.

let $Q = \{q_0, q_1, q_2 \dots q_n\}$

let $G = (V, \Sigma, P, S)$.

$V = \{A_{q_0}, A_{q_1}, A_{q_2} \dots A_{q_n}\}$

$S = A_{q_0}$

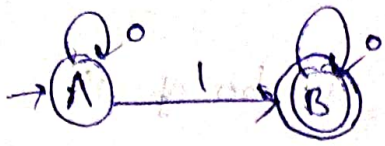
If $\delta(q_i, a) = q_j$ then we have a P.E

$$P_{q_i} \rightarrow a q_j$$

* In addn if q_j is final state we have

$$A_{q_i} \rightarrow a$$

Eg: Generate CFG for the RE $0^*1(0+1)^*$



G_6:

$A \to 0A$

$A \to 1B$

$B \to 0B$

$B \to 1B$

$A \to 1$

$B \to 0$

$B \to 1$

G_7: $S \to aS/bS/a/b/\varepsilon$ $\quad \Sigma^* / (a+b)^*$

G_8: $S \to SS/[S]/(S)/[]/()$ Balanced parenthesis on $\Sigma = \{[,],(,)\}$

G_9: $SaSbS/SbSaS/\varepsilon$ Equal number of a's & b's

* Derivations & parse trees:

G_3: 1 $S \to aB$

2 $S \to bA$

3 $A \to aS$

4 $A \to bAA$

5 $A \to a$

6 $B \to bS$

7 $B \to aBB$

8 $B \to b$

odd no. of a's & odd no. of b's

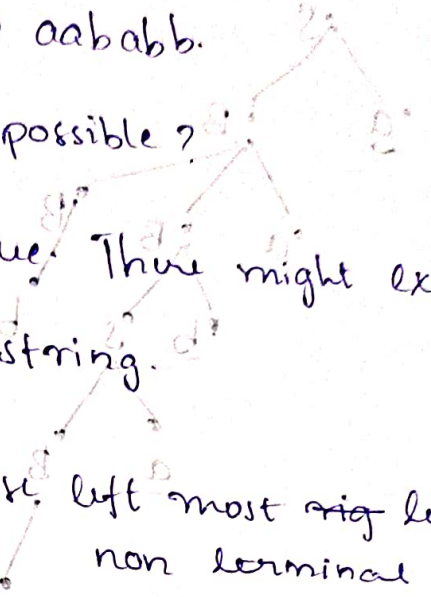
aabaabb:

$S \Rightarrow aB \Rightarrow aaBB \Rightarrow aaBb \Rightarrow aabSb \Rightarrow aababB$

$\Rightarrow aababb.$

is it the only derivation possible?

- Derivations are not unique. There might exist several derivations for a string.



* left most derivation (choose left most ~~rig~~ left most non terminal)

$S \Rightarrow aB \Rightarrow aaBB \Rightarrow aabSB \Rightarrow aababB$

$\Rightarrow aababB \Rightarrow aababb.$

* right most derivation

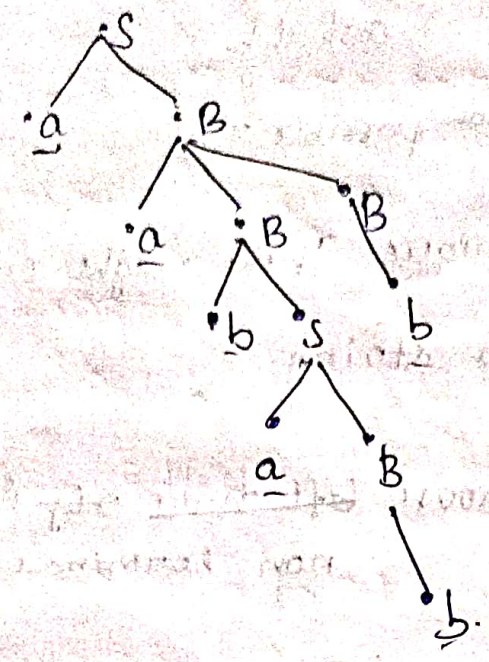
$S \Rightarrow aB \Rightarrow aaBB \Rightarrow aaBaBB \Rightarrow aaBaBb$

$\Rightarrow aaBabb \Rightarrow aababb$

* left most derivation & right most derivations are unique

→ If there is a derivation then unique LMD & unique RMD both will be there.

→ Parse trees:



→ Root is Start symbol.

→ Internal nodes are labelled as non terminals

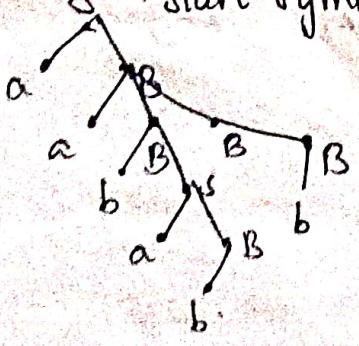
→ leaf nodes are labelled as terminals.

→ Read leaf nodes from left to right to get the ~~str~~ terminal string.

→ For a given derivation, there is only one parse tree but for a given parse tree there may be more than one derivation

→ for a given parse tree, there exists a unique left most derivation and a unique right most derivation

* Graphical Representation of parse trees:



S → Start symbol

→ Internal nodes are non terminal $\in V$

→ leaf nodes are terminals $\in \Sigma$

• Read all leaf nodes from left to right gives terminal string (or) yield $\in \Sigma^*$

* Ambiguous grammar

(not clear) G_{10} generates expressions with +/*

G_{10} : $E \to E+E$

$E \to E*E$

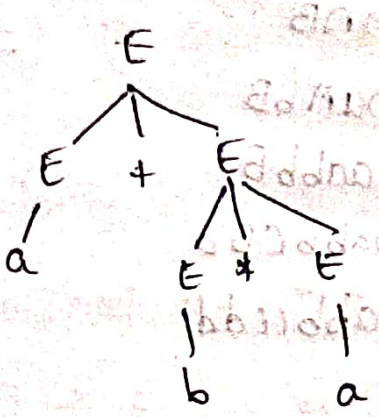
$E \to a$

$E \to b$

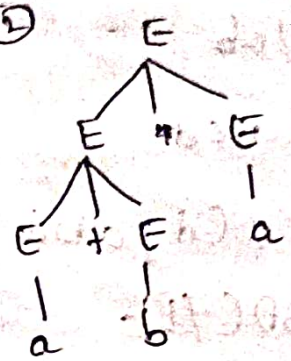
a+b*a

$S \Rightarrow a+b*a$

①



②



① & ② There are 2 derivations for the string.

→ Which path to get terminal string? (ambiguity)

CFG is ambiguous for a given string in lang for which there are more than 1 parse trees.

→ Finding a grammar ambiguous / not is an undecidable problem

→ In many situations, an ambiguous grammar can be converted to un-ambiguous grammar

★ (should have unique non-terminals in production rules).

G_{10}' : $E \to (E+T) / T$

$T \to T*F / F$

$F \to a / b$

A CFL is inherently ambiguous if every CFG that generates the language is an ambiguous language.

$L = \{a^n b^n c^m d^m \mid n \geq 1, m \geq 1\} \cup \{a^n b^m c^m d^n \mid n \geq 1, m \geq 1\}$

↳ ambiguous language

G_{11}: $S \to AB / C$

$A \to aAb / ab$

$B \to cBd / cd$

$C \to aCd / aDd$

$D \to bDc / bc$

aabbccdd

① $S \Rightarrow AB$

$\Rightarrow aAbB$

$\Rightarrow aabbB$

$\Rightarrow aabbcBd$

$\Rightarrow aabbccdd$

② $S \Rightarrow C$

$\Rightarrow aCd$

$\Rightarrow aaDdd$

$\Rightarrow aabDcdd$

$\Rightarrow aabbccdd$

G_{12}: $S \to aS / aSbS / \varepsilon$ aab.

① $S \Rightarrow aS$

$\Rightarrow aaSbS$

$\Rightarrow aab$

② $S \Rightarrow aSbS$

$\Rightarrow aaSbS$

$\Rightarrow aab$

* Simplification / minimization of CFG.

$G = (V, T, P, S)$ & $G' = (V', T', P', S)$

G' will be simplification of G iff $L(G) = L(G')$

→ G & G' are equivalent

Step (1) : Removal of useless symbols

Step (2) : Removal of epsilon productions

Step (3) : Removal of unit productions.

(1) Removal of useless symbols.

→ A symbol is useless if it doesn't take part in the derivation of some string in the language

(1.1) Removal of non-generating symbols (ngs)

→ A non-terminal symbol A can be useless if there is no $w \in \Sigma^*$ such that $A \Rightarrow^* w$

$S \Rightarrow^* \alpha A \beta \dashrightarrow w$ here A is a ngs.

$S \to AB$
$A \to BC/a$
$B \to b$
$C \to DE$
$D \to d$
$E \to e$

C → not generating symbol.

{a, b, d, e, A, B, D, E, S, A}

Identification of ngs.

→ Let G_1 be the set of generating non-terminals.

We know that A is generating if $A \Rightarrow^* w \in \Sigma^*$

Basis : $A \to w$, $w \in \Sigma^*$

Add all $A \in V$ such that $A \to w \in \Sigma^*$ is in P.

Eg: 1) $S \to AB$ $G = \{A, C, D\}$ ← "things deriving terminals"

i) $A \to a$
$B \to CD$
$C \to C$
$D \to d$

$G = \{A, B, C, D\}$
$= \{S, A, B, C, D\}$
↗ All symbols (no NGS).

ii) $S \to AB$
$A \to a$
$B \to CD$
$D \to d$

$G = \{A, D\}$ → so B, C, S are ngs ✓
✓ useful symbol → PR with no useful symbol becomes useless

Induction: $B \to \alpha$, if every non terminal in α (PR) is already in G, then add B to G, if B is not already in G.

Set of non-generating symbols is V-G.

1.2

(urs)
Identifying un-reachable symbol

From start state you cannot reach the symbol ✓ urs.

→ urs can be both terminal & non-terminal. A symbol (terminal / non-terminal) is a urs if there is no way of reaching the symbol from the start state that is suppose we have $S \not\Rightarrow \alpha A \beta$

$\forall \alpha, \beta \in (V \cup \Sigma)^*$

$S \to AB$ ✗
$A \to CD$
$C \to c$
$E \to FG$
$F \to f$
$G \to g$
$D \to d$

$R = \{S\}$

$R = \{S, A, B\}$

$R = \{S, A, B, C, D\} \cup \{e\} \cup \{d\}$

$urs = V - R$

Question 1.1 / 1.2 first?

Eg: $S \to AB$ ✗ remove ngs. $\Rightarrow$ $S \to a$, $A \to b$ $\Rightarrow$ remove urs $\Rightarrow$ $S \to a$ ✓

$S \to a$
$A \to b$
$\{a, b, A, S\}$

(B)

Eg: $S \to AB$, $S \to a$, $A \to b$ $\Rightarrow$ remove urs $\Rightarrow$ $S \to AB$, $S \to a$, $A \to b$ $\Rightarrow$ remove ngs $\Rightarrow$ $S \to a$, $A \to b$ — urs still there

* So always first apply ① ngs ⟶ ② urs then

---

9) $S \to AB \mid AC$ ✗
$A \to aAb \mid bAa \mid a$
$B \to bbA \mid aaB \mid AB$
$C \to abCA \mid aDb$ } useless
$D \to bD \mid aC$ } PR

useful = {a, b, A, B, S}
useless → C, D

Step ② : Removal of ε-productions.

We know $A \rightarrow \varepsilon$ is called as ε-production.

Let $G = (V, T, P, S)$ if $L(G)$ includes $S \overset{*}{\Rightarrow} \varepsilon$ then clearly it is not possible to obtain a grammar G' without any ε-productions

$\forall\ L(G') = L(G)$.

Continuation -

Removal of ε-productions

Given a grammar G to obtain G' such that

$$L(G') = L(G) \setminus \{\varepsilon\}$$

Identify all nullable non-terminal.

We say A is nullable if $A \overset{*}{\Rightarrow} \varepsilon$, if $A \Rightarrow \varepsilon$ is a production rule, clearly A is nullable.

Basis: Let N be the set of nullable non-terminals.

$N = \{A \in V \mid A \to \varepsilon \text{ is a production rule}\}$

Induction:

Suppose $A \in N$, $B \in N$, $C \in N$

Suppose $D \to ABC$

Formally $A \to \alpha$, α consists of nullable symbols then add A to N, if A is not already in N.

Given $G = (V, T, P, S)$ & N set of nullable non-terminals, to obtain G' such that

$$L(G') = L(G) \setminus \{\varepsilon\}$$

Step 1: Eliminate all ε-productions from P.

Step 2: If $A \to X_1, X_2 \cdots X_k$ is in P_i, and one or some of these X_i's is/are in N then add all productions of the form

$A \to Y_1 Y_2 \cdots Y_k$ except $A \to \varepsilon$.

where $y_i = \begin{cases} x_i \\ \varepsilon \text{ if } x_i \in N \end{cases}$

$S \to AB$
$A \to aAA \mid \varepsilon$
$B \to bBB \mid \varepsilon$

$A \to \varepsilon$
$B \to \varepsilon$

To remove $A \to \varepsilon$ replace each occurrence of $A \in N$, $B \in N$ $C \to AB$ then $C \in N$ of A with ε then $S \in N$ so $N = \{A, B, S\}$

Step 1 ⇓

$S \to AB$
$A \to aAA$
$B \to bBB$

Step 2 ⟹

$S \to AB \mid B \mid A$
$A \to aAA \mid aA \mid a$
$B \to bBB \mid bB \mid b$

* dont include if we get ε again

3) Elimination of unit productions:

$A \to B$ (non terminal, non terminal)

→ Unit productions are of the form $A \to B$, that is there is exactly one non terminal at the RHS of a production rule

→ Given G to obtain G' such that $L(G) = L(G')$ and G' does not contain unit productions.

Let U be the set of unit productions ~~add~~ add

Basis: All pairs (A, B) satisfying $A \to B$ to U

Induction: $(A, B) \in U$ & $B \to C$ is in P then add (A, C) to U;

To obtain G' without unit productions

$L(G') = L(G) \setminus \{\text{unit productions}\}$

Step 1 : Remove all unit productions.

Step 2 : Add new production rules.

Suppose $A \overset{*}{\Rightarrow} B$ & $B \to \alpha$ is a non unit production, then add in G, the new unit production $A \to \alpha$

Example:

$I \to a \mid b \mid Ia \mid Ib \mid I0 \mid I1$

$F \to I \mid (E)$

$T \to F \mid T * F$

$E \to T \mid E + T$

Non-unit

$I \to a \mid b \mid Ia \mid Ib \mid I0 \mid I1$

$F \to (E)$

$T \to T * F$

$E \to E + T$

Base case:

unit productions

(E, E)
(E, T)
(E, F)
(E, I)
(T, T)
(T, F)
(T, I)
(F, F)
(F, I)
(I, I)

~~(F, F)~~ $E \to E + T$ ①

(F, I) $E \to T * F$ ②

~~(I, I)~~ $E \to (E)$ ③

$E \to a \mid b \mid Ia \mid Ib \mid I0 \mid I1$ ④

$T \to T * F$ ⑤

$T \to (E)$ ⑥

$T \to a \mid b \mid Ia \mid Ib \mid I0 \mid I1$ ⑦

$F \to (E)$ ⑧

$F \to a \mid b \mid Ia \mid Ib \mid I0 \mid I1$ ⑨

$I \to a \mid b \mid Ia \mid Ib \mid I0 \mid I1$ ⑩

new grammar with no unit productions

* Safe ordering → removal of ε-productions
→ removal of unit productions.
→ removal of useless symbols.

→ 1.1
→ 1.2

* Chomsky normal form (CNF):

A CFG is said to be in CNF, if every production rule is one of the following types.

*(i) $A \to a$, $a \in T$ i.e → on rhs of pr, only 1 terminal (or)

*(ii) $A \to BC$, $A, B, C \in V$ 2 non terminals

By doing this Parse tree → Binary tree

* A CFG 'G' to be in CNF, It has to be simplified version of G.
* i.e → we assume G has no useless symbols, no ε productions, no unit productions

Conversion of Simplified CFG G' to CNF.

Step(1) → Ensure that RHS of every PR consists of only non-terminals or a single terminal.

Ex:- $A \to BaC \Rightarrow A \to ABDC$
$D \to a$

Step(2) → Make sure every RHS of a production has exactly 2 Non terminals / single terminal

Ex:- $A \to BDC \Rightarrow A \to BE$
$D \to a$ $E \to DC$
$D \to a$

Examples:-

(i) G_{13} : $S \to ASB \mid \varepsilon$
$A \to aAS \mid a$
$B \to sbs \mid A \mid bb$

removing ε prod ⟹

$N = \{S\}$
$S \to ASB \mid AB$
$A \to aAS \mid aA \mid a$
$B \to sbs \mid bs \mid Ss \mid a \mid bb \mid b$

removing unit production ⟹ (identify Single non-T rns pair) of A (non-unit productions)

(B, A) $B \to aAS \mid aA \mid a$

(S, S) $S \to ASB \mid AB$

(A, A) $A \to aAS \mid aA \mid a$

(B, B) $B \to SbS \mid bS \mid Sb \mid b \mid bb$

removal of ngs ⟹ generating symbol set

~~X~~ = {A, B} {no ngs}

X = {S, A, B} continue

removal of urs ⟹ R = {S}

R = {S, A, B} (same)

CNF Step (1) $S \to ASB \mid AB$ ①

$A \to A, AS$ ② $A \to A, A$ ③ $A \to a$ ④

$A_1 \to a$ ⑤

$B \to A, AS$ ⑥ $B \to A, A$ ⑦ $B \to a$ ⑧

$B \to SB, S$ ⑨ $B \to B, S$ ⑩ $B \to b$ ⑪

$B_1 \to b$ ⑫ $B \to SB_1$ ⑬ $B \to B, B_1$ ⑭

CNF Step (2) ⟶

(i) $S \to ASB$ $S \to A, B$ $S_1 \to AS$ $S \to AB$

(2) $A \to A, AS$ $A \to A, S_1$

(3) $A_1 \to a$ (4) $A \to A, A$ (5) $A \to a$

(6) $B \to A, AS \Rightarrow B \Rightarrow A, S_1$ (7) $B \to A, A$ (8) $B \to a$

(9) $B \to SB, S \Rightarrow B \to B_2 S$ $B_2 \to SB_1$ (10) $B_1 \to b$

$B \to b$ rest all ok ✓ (Grammer in CNF)

9. $S \to aY \mid YbbY$

$X \to a \mid \varepsilon$ $\Longrightarrow$ $N = \{X\}$

$Y \to aYY \mid bb \mid XXa$

$S \to aY \mid YbbY$

$X \to a$

$Y \to aYY \mid bb \mid XXa \mid Xa \mid a$

(S,Y)

(S,S)

(X,X)

(Y,Y)

$S \to aY \mid YbbY \mid aYY \mid bb \mid XXa \mid Xa \mid a$

$X \to a$

$Y \to aYY \mid bb \mid XXa \mid Xa \mid a.$

CNF :

Step-1 :

$S \to aY$	$S \to Y_1Y$ $Y_1 \to a$
$S \to YbbY$	$S \to YY_2Y_2$ $Y_2 \to b$
$S \to aYY$	$S \to Y_1YY$
$S \to bb$	$S \to Y_2Y_2$
$S \to XXa$	$S \to XXY_1$
$S \to Xa$	$S \to XY_1$
$S \to a$	$S \to a$
$X \to a$	$X \to a$

$Y \to aYY$	$Y \to Y_1YY$
$Y \to bb$	$Y \to Y_2Y_2$
$Y \to XXa$	$Y \to XXY_1$
$Y \to Xa$	$Y \to XY_1$
$Y \to a$	$Y \to a$

Step - 2 make sure to have 2 non terminals

$S \to Y_1 Y$

$Y_1 \to a$

$S \to Y Y_3$

$Y_3 \to Y_2 Y_2$

$Y_2 \to a$

$S \to Y_1 Y_4$

$Y_4 \to Y Y$

$S \to Y_2 Y_2$

$S \to X_1 Y_1$

$X_1 \to X X$

$S \to X Y_1$

$S \to a$

$X \to a$

$Y \to Y_1 Y_4$

$Y \to Y_2 Y_2$

$Y \to X_1 Y_1$

$Y \to X Y_1$

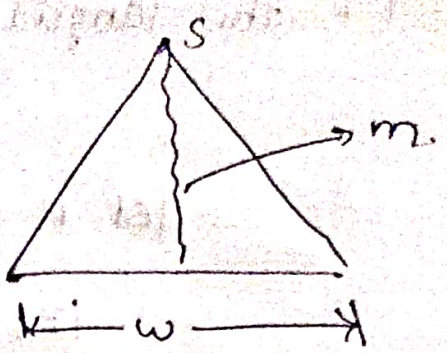
$Y \to a$

* Suppose G is a CNF of a CFG. Consider a derivation tree of G, generating terminal string w such that no path in derivation tree has length > m then $|w| \leq 2^{m-1}$

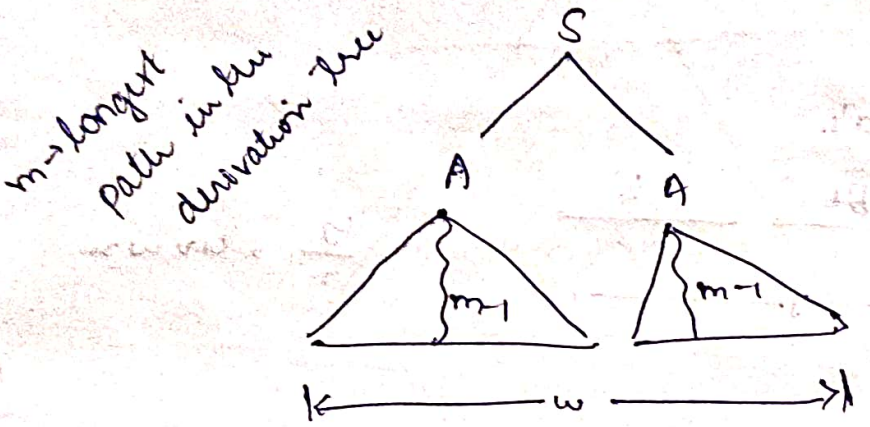
Proof by induction:

Basis: $m > 1$

$|w| \leq 2^{1-1} = 1.$



Induction:



$2 * 2^{m-1}$

$|w| \leq 2^m$ → one less than the longest

$|w| \leq 2^{m-1}$

Pumping lemma for CFLs

Let L be a CFL. Then there exists a constant n, such that if z is any string in L such that $|z| \geq n$, then we can write $z = uvwxy$ satisfying

i) $|vwx| \leq n$

ii) $|vx| \neq 0$ that is $vx \neq \varepsilon$

iii) for all $i \geq 0$, $uv^iwx^iy \in L$

Eg: PT the language $L = \{a^n b^n c^n \mid n \geq 0\}$ is not CFL.

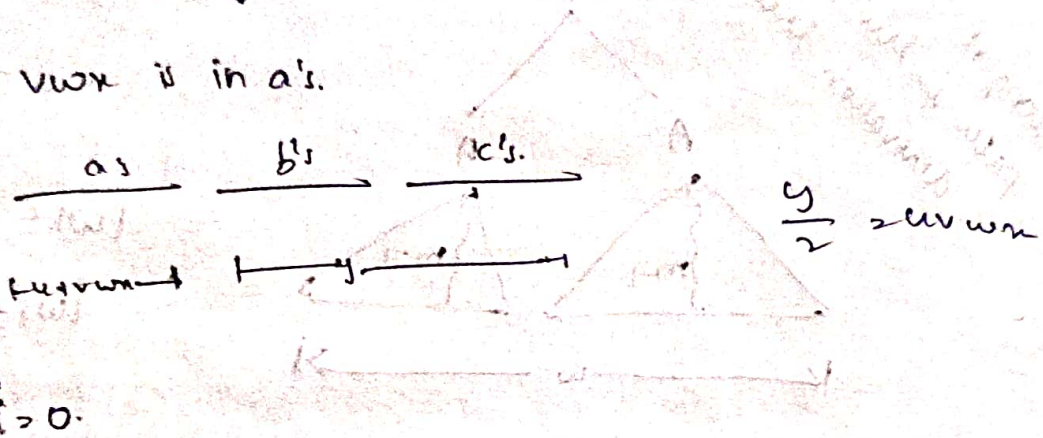
let k be the pumping constant

$|z| \geq k$.

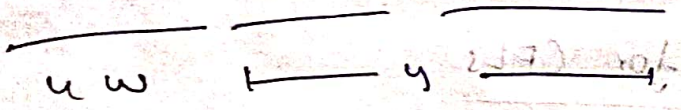
$z = a^k b^k c^k$, $|z| = 3k \geq k$

uvwxy

Case 0: vwx is in a's.



for $i = 0$.



but we get $\frac{y}{2} > uw$

contradiction

Therefore not CFL.

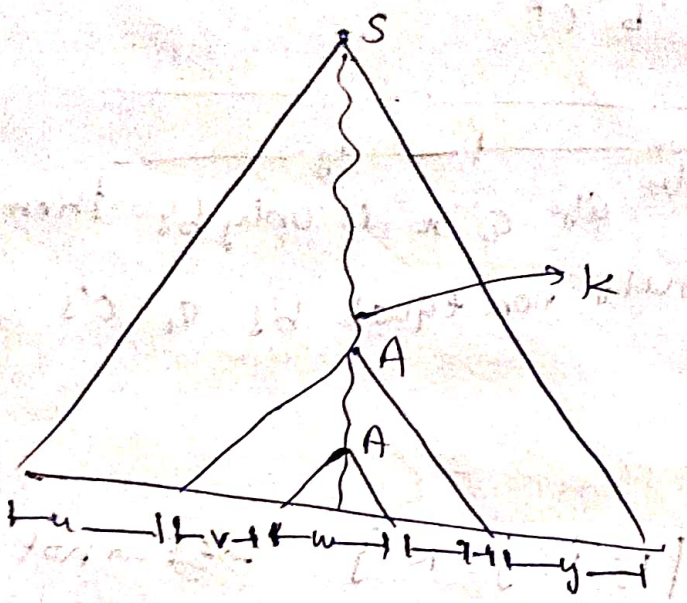
* Proof:

Fact: Suppose G is a CNF grammar. Let T be a derivation tree of G & let T have no path longer than m (path from root to leaf). Then the string generated is of length $\leq 2^{m-1}$

→ let G be a CNF grammar

→ let G has m unique non-terminals

→ Consider a derivation tree of G of a string in L whose length $\geq 2^m$.



$A \overset{*}{\Rightarrow} w$

$A \overset{*}{\Rightarrow} vAx$

$S \overset{*}{\Rightarrow} uvwxy$

$S \overset{*}{\Rightarrow} uAy$

$S \Rightarrow uvAxy.$

$S \overset{*}{\Rightarrow} uAy.$

$\Rightarrow uvAxy$

$\overset{*}{\Rightarrow} uvvAxxy$

$\overset{*}{\Rightarrow} uvvv \cdots Axxx \cdots xy \Rightarrow uviwxiy$ — (3)

$|vx| \neq 0$ or $vx \neq \varepsilon$ — (2)

Assume $v = \varepsilon$ & $x = \varepsilon$ and substitute in $A \overset{*}{\Rightarrow} vAx$, we will get $A \overset{*}{\Rightarrow} A$. This is a contradiction as G is CNF.

(1) $|vwx| \leq n$.

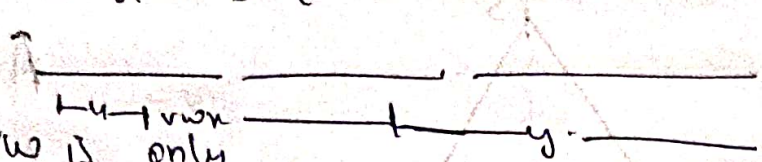
$|vwx| \leq 2^{n-1} < 2^{k-1} = n$

$L_2 = \{a^m b^n c^n \mid n \geq m \geq 1\}$

Let k be a pumping constant

we need to take a z such that $|z| \geq k$

$a^{k-1} b^k c^k$



If vw is only a's & x is only b's then pumping down result non equal b's & c's which is contradiction

$L_3 = \{a^i b^j c^k \mid 1 \leq i \leq j \leq k\}$ → not a CFL

$L_4 = \{a^m b^n c^m d^n \mid m, n \geq 1\}$ → not a CFL

$L_5 = \{a^m b^m c^n d^m \mid m, n \geq 1\}$ → CFL

$L_6 = \{a^m b^n c^n d^m \mid m, n \geq 1\}$ → CFL (push, push, pop, pop)

Closure properties of CFL:

1) The class of CFL is not closed under intersection.
2) is closed under union
3) is not closed under complementation
4) is closed under concatenation
5) is closed under kleene star
6) is closed under reversal
7) CFLs are closed when intersected with RLs.

8) The class of CFLs are not closed under set difference

9) Set difference with RL is CFL.

① → Proof by counter example

$$L_1 = \{a^i b^i c^j \mid i, j \geq 1\}$$

$$L_2 = \{a^i b^j c^j \mid i, j \geq 1\}$$

$$L_1 \cap L_2 = \{a^n b^n c^n \mid n \geq 1\} \rightarrow \text{not a CFL.}$$

② → Proof by construction

$$G_1 = (V_1, \Sigma, P_1, S_1) \quad G_2 = (V_2, \Sigma, P_2, S_2)$$

$L_3 = L(G_1) \cup L(G_2)$

$$G = (V, \Sigma, P, S)$$

$$V = \{V_1\} \cup \{V_2\} \cup \{S\}$$

$$P = \{P_1\} \cup \{P_2\} \cup \{S \rightarrow S_1 \mid S_2\}$$

③ → Proof by contradiction

let $L_1 \cap L_2 = \overline{(\overline{L_1} \cup \overline{L_2})}$ → Apply demorgan's law

let $\overline{L_1}$ is a CFG & $\overline{L_2}$ is a CFG, then $\overline{\overline{L_1} \cup \overline{L_2}}$ is a CFG.

then $L_1 \cap L_2$ should also be a CFG. But we already know $L_1 \cap L_2$ is not a CFG. Therefore we arrived at a contradiction that CFL is not closed under complementation

④ → Proof by construction

$G_1(V_1, \Sigma, P_1, S_1)$ $G_2(V_2, \Sigma, P_2, S_2)$

$L_3 = L(G_1) \cap L(G_2)$

$G = (V, \Sigma, P, S)$

$V = \{V_1\} \cup \{V_2\} \cup \{S\}$

$P = \{P_1\} \cup \{P_2\} \cup \{S \to S_1 S_2\}$

⑤ → Proof by construction

let L be a CFL with grammar $G = (V, \Sigma, P, S)$ then L^R is also CFL with grammar.

$G = \{V \cup \{S\}, \Sigma, P \cup \{S \to S, S | \varepsilon\}, S\}$.

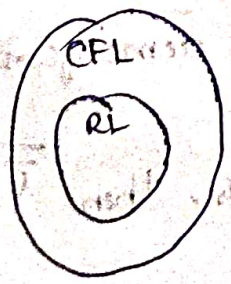
⑥ If $A \to \alpha$ is a production in G, then

$A \to \alpha^R$ is a production in G^R

Eg: If $A \to BC$ PR in G then

$A \to CB$ is in G^R

⑦



(8) $L_1 - L_2 = L_1 \cap \bar{L_2}$

Since $\cap$ is not ~~closed~~ a CFL so set difference is also not a CFL.

(9) $L_1 - L_2$
CFL RL

let L_1 is a CFL & L_2 is RL

$L_1 - L_2 = L_1 \cap \bar{L_2}$

we know complement of a RL is RL. so $\bar{L_2} \rightarrow$ RL.

we also know CFLs are closed under intersection with RL. Therefore $L_1 \cap \bar{L_2}$ is a CFL. Hence $L_1 - L_2$ is also a CFL.

* Decision problem for CFLs:

1) Given a CFG G, is ~~L(G) ≠ ∅?~~ $L(G) = \emptyset$?

if the start symbol do not generate any terminal strings, then that language is empty.

To do this, we have already discussed an algo to find generating symbols. At the end of the algo if the set of generating symbols is empty, then the language is empty.

The time complexity of the algorithm to generate set of generating symbols is $O(n^2)$ which is a polynomial time.

2) Given a grammar is L(G) infinite?

Algorithm

Step 1 : Find CNF for G to get

$$L(G_1) = L(G) - \{\varepsilon\}$$

Is $L(G_1)$ infinite ?

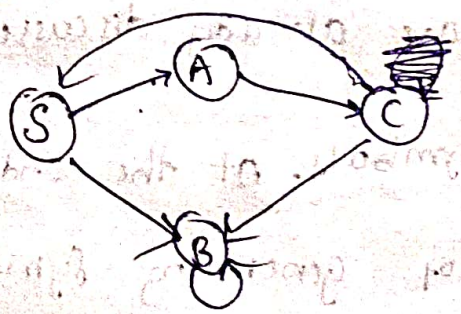
Step 2 : Create a directed graph $g = (V_0, E)$ such that g has non-terminals of G_1 as vertices.

From a nonterminal A in g, there is a edge to the non-terminal B iff there is a production rule $A \to BC$ or $A \to CB$ for some non terminal C.

Claim : If g has a cycle, then $L(G_1)$ is infinite & vice versa.

Example : $S \to AB$, $A \to acb / a$, $B \to bB / b$

$C \to cBS$



3) Given a CFG G & a string x, is $x \in L(G)$

Brute-force :

→ WKT if a grammar is in CNF, we can derive a string in $(2|w|-1)$ steps

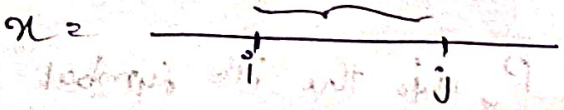
→ List all the possible parse trees with $(2|w|-1)$ steps & check any of the parse tree is generating required string.

→ But this is an exponential time algorithm.

→ There is an efficient algorithm based on DP.

* CYK algorithm / Tabulation algorithm

free code camp
greedy & DP

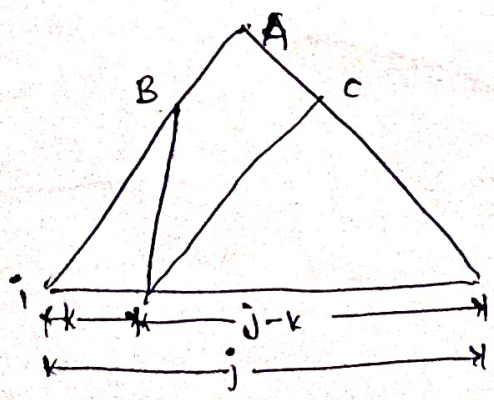


$$V_{ij} = \{ A \in V_N \mid A \overset{*}{\Rightarrow} x_{ij} \}$$

$x \in L(G)$, iff V_{1n} contains S where $|x| = n$.

We can define V_{ij} provided, we already have the definitions of V_{lk}, $k < j$, for all l.

$V_{11} = \{ A \mid A \to a$ is a production rule where a is the ith symbol of $x \}$.



$A \in V_{ij}$ iff $A \overset{*}{\Rightarrow} x_{ij}$

we know in CNF, we have $A \to a$ & $A \to BC$

$B \in V_{ik}$

$C \in V_{i+1k, j-k}$ $\quad 1 \le k \le j-1$

Re write as

$A \in V_{ij}$ iff there exist B & C such that $B \in V_{ik}$ & $C \in V_{i+1k, j-k}$ for all $1 \le k \le j-1$ and $A \to BC$ is a PR in G.

Algo

for i = 1 to n

$V_{11} = \{A \mid A \to a$ is in P, a is the ith symbol in x$\}$

for j = 2 to n {

for i = 1 to n−j+1 {

$V_{ij} = \phi$

for k = 1 to j−1 {

$V_{ij} = V_{ij} \cup \{A \mid A \to BC$ is in P &

$B \in V_{ik}$ and $C \in V_{i+1k, j-k}\}$

}

}

TC → $O(n^3)$

for a given grammar

different from

$O(n^3)$

Example:

G: $S \to AB \mid BC$
$A \to BA \mid a$
$B \to CC \mid b$
$C \to AB \mid a$

Check that "baaba" belongs to G

Photo in phone

	5	4	3	2	1
1	A,C	ϕ	ϕ	{A,S}	B
2	A,C	B	B	{A,C}	
3	B	{S,C}	{A,C}		
4	{A,S}	B			
5	{A,C}				

$V_{1n} \Rightarrow x$

V_{15}

15 → has start symbol

So the string belongs to the language.

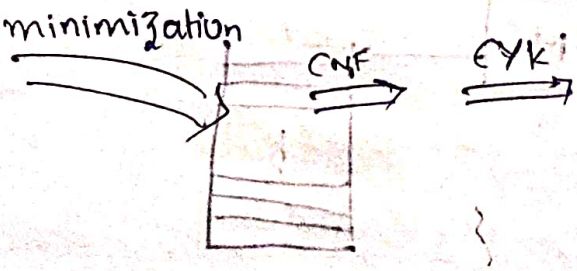
Example:

$S \to AB \mid XB \mid \varepsilon$
$T \to AB \mid XB$
$X \to AT$
$A \to a$
$B \to b$

not in CNF

is $w = aaabb$ in $L(G)$

is $w = aaabbb$ in $L(G)$



* Push Down Automata (PDA):

CFG / PDA

→ The class of PDAs recognise the class of CFLs.

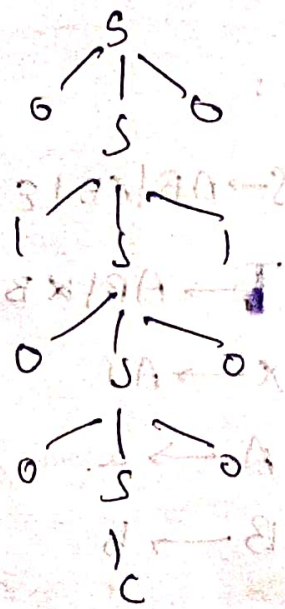
→ A CFG is generative, i.e a string is generated by a grammar.

→ A PDA accepts / rejects a string.

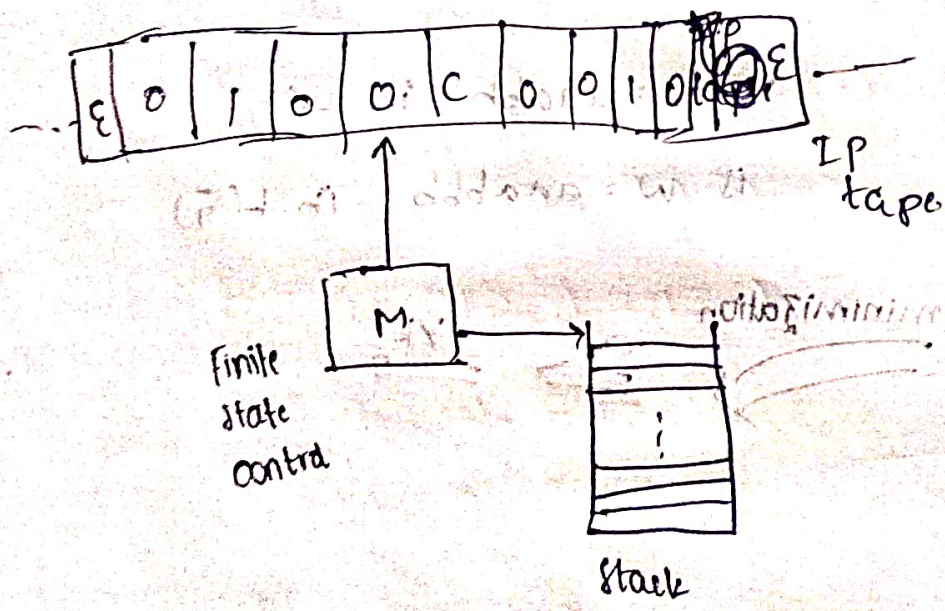
→ For every PDA, there is exactly one CFL that is defined by that PDA.

Eg: $L = \{wcw^R \mid w \in \{0,1\}^*, c \notin \{0,1\}\}$

$S \rightarrow 0S0 \mid 1S1 \mid c$

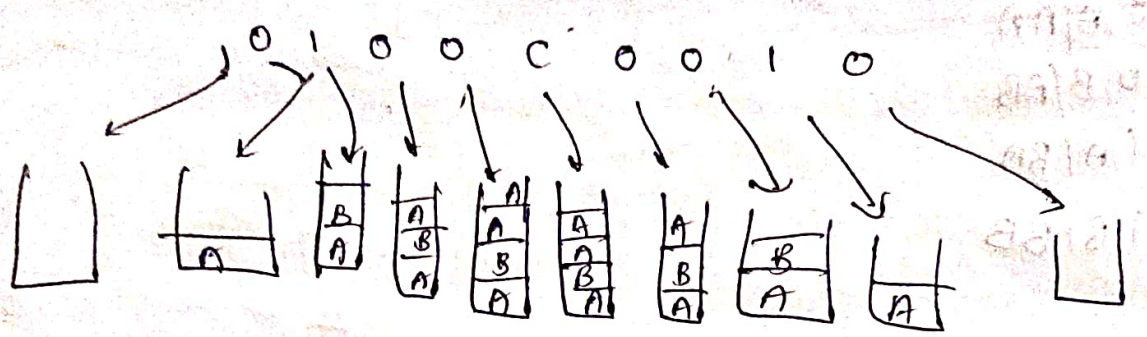


→ A PDA will have a stack / pushdown store.



Correspondence

ip symbol	stack symbol
0	A
1	B



* PDA is a 7-tuple machine.

$(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$

- Q: finite set of states
- Σ: input alphabets
- Γ: stack alphabets
- δ: transition fkn.
- q_0: initial state eg
- Z_0: start symbol (stack)
- F: set of final states

$$\delta : Q \times \Sigma \times \Gamma \longrightarrow Q \times \Gamma^*$$

$a, AA \rightarrow$ push
$q, \varepsilon \rightarrow$ pop
$q, A \rightarrow$ no op

$\delta(q_0, 0, Z_0) \vdash (q_0, AZ_0)$

$\delta(q_0, 1, Z_0) \vdash (q_0, BZ_0)$

$\delta(q_0, 0, A) \vdash (q_0, AA)$

$\delta(q_0, 0, B) \vdash (q_0, AB)$

$\delta(q_0, 1, A) \vdash (q_0, BA)$

$\delta(q_0, 1, B) \vdash (q_0, BB)$

$\delta(q_0, C, A) \vdash (q_1, A)$

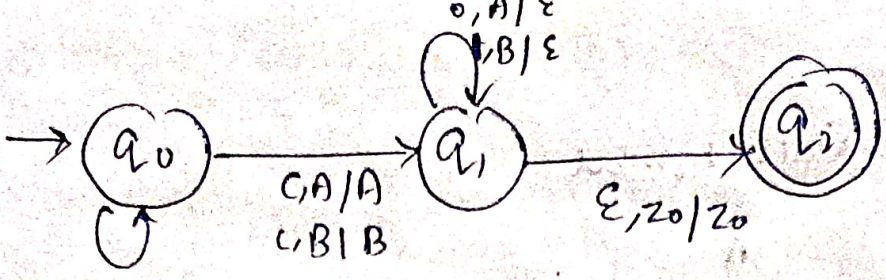
$\delta(q_0, C, B) \vdash (q_1, B)$

$\delta(q_1, 0, A) \vdash (q_1, \varepsilon)$

$\delta(q_1, 1, B) \vdash (q_1, \varepsilon)$

$\delta(q_1, \varepsilon, Z_0) \vdash (q_2, Z_0)$

$\delta(q_0, C, Z_0) \vdash (q, Z_0)$



0, Z0 / AZ0
1, Z0 / BZ0
0, A / AA
0, B / AB
1, A / BA
1, B / BB

Eg: $\delta(q_0, 0100c0010, Z_0)$

$\vdash \delta(q_0, 100c0010, AZ_0)$

$\vdash \delta(q_0, 00c0010, BAZ_0)$

$\vdash \delta(q_0, 0c0010, ABAZ_0)$

$\vdash \delta(q_0, c0010, AABAZ_0)$

$\vdash \delta(q_1, 0010, AABAZ_0)$

$\vdash \delta(q_1, 010, ABAZ_0)$

$\vdash \delta(q_1, 10, BAZ_0)$

$\vdash \delta(q_1, 0, AZ_0)$

$\vdash \delta(q_1, \varepsilon, Z_0)$

$\vdash \delta(q_2, \varepsilon, Z_0)$

→ Acceptance by final state

→ Acceptance by empty stack

* DPDA : $\delta : Q \times \Sigma \times \gamma \longrightarrow (P, \gamma)$

DPDA $\subseteq$ NPDA

so NPDA more powerful

NDPDA : $\delta : Q \times \Sigma \times \gamma \longrightarrow \{(P_1, \gamma_1), (P_2, \gamma_2) \dots (P_k, \gamma_k)\}$

$\delta : Q \times \Sigma \cup \{\varepsilon\} \times \gamma \rightarrow \{(P_1, \gamma_1), (P_2, \gamma_2) \dots (P_k, \gamma_k)\}$

$\delta : Q \times \Sigma \cup \{\varepsilon\} \times \Gamma \longrightarrow P(Q \times \Gamma^*)$

power set.

Eg : $L = \{\omega\omega^R \mid \omega \in \{0,1\}^*\}$

Machine has to guess.

$\delta(q_0, 0, z_0) = (q_0, Az_0)$

$\delta(q_0, 1, z_0) = (q_0, Bz_0)$

$\delta(q_0, 0, A) = (q_0, AA)$

$\delta(q_0, 0, B) = (q_0, AB)$

$\delta(q_0, 1, A) = (q_0, BA)$

$\delta(q_0, 1, B) = (q_0, BB)$

$\delta(q_0, \varepsilon, A) = (q_1, A)$

$\delta(q_0, \varepsilon, B) = (q_1, B)$

$\delta(q_1, 0, A) = (q_1, \varepsilon)$

$\delta(q_1, 1, B) = (q_1, \varepsilon)$

$\delta(q_1, \varepsilon, z_0) = (q_2, z_0)$

$\delta(q_0, \varepsilon, z_0) = (q_1, z_0)$

* Instantaneous description (ID)

Imagine that a PDA is working and at a certain step, we halt it.

→ The information need to restart a PDA once it is halted is called ID.

→ If ~~we~~ three pieces of information (current state q, i/p yet to be processed w, current stack contents γ) is available then we can restart the PDA & see how this PDA will evolve in future.

(q, w, γ)

$ID_1 \vdash ID_2 \vdash ID_3 \vdash \cdots \vdash ID_k$.

$ID_1 \overset{*}{\underset{M}{\vdash}} ID_k$ → 0 or more no. of steps

* $(q, aw, xy) \underset{M}{\vdash} (p, w, \beta y)$

iff there exist $\delta(q, a, x) = (p, \beta)$

$\vdash$ → deriving eq

$=$ → used for δ

& Acceptance by ~~the~~ empty stack eg:

$L = \{0^n 1^n \mid n \geq 0\}$ 00011,

$\delta(q_0, 0, Z_0) = (q_0, AZ_0)$

$\delta(q_0, 0, A) = (q_0, AA)$

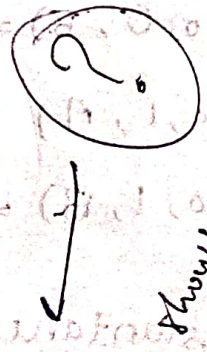
$\delta(q_0, 1, A) = (q_0, \varepsilon)$

~~$\delta(q_0, \varepsilon, Z_0) = (q_0, \varepsilon)$~~

$\delta(q_1, 1, A) = (q_1, \varepsilon)$

$\delta(q_1, \varepsilon, Z_0) = (q_1, \varepsilon)$

$\delta(q_0, \varepsilon, Z_0) = (q_1, \varepsilon)$



any state should be present in acceptance by empty stack

L: {w ∈ {0,1}* / w has equal number of 0's & 1's}

$\delta(q_0, 0, Z_0) = (q_0, AZ_0)$

$\delta(q_0, 0, A) = (q_0, AA)$

$\delta(q_0, 1, Z_0) = (q_0, BZ_0)$

$\delta(q_0, 1, B) = (q_0, BB)$

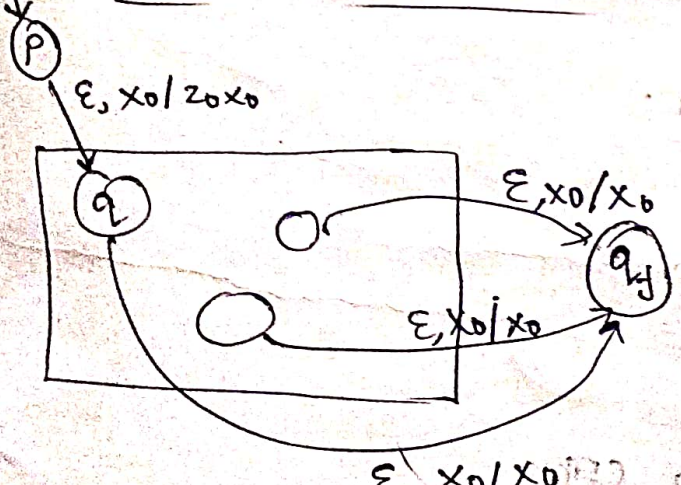
$\delta(q_0, 0, B) = (q_0, \varepsilon)$

$\delta(q_0, 1, A) = (q_0, \varepsilon)$

$\delta(q_0, \varepsilon, Z_0) = (q_0, \varepsilon)$ → Acceptance by empty stack.

* Equivalence of acceptance by final state & acceptance by empty stack:-

1) Acceptance by empty stack to acceptance by final state:



$M = \{Q, \Sigma, \Gamma, \delta, q_0, Z_0\}$

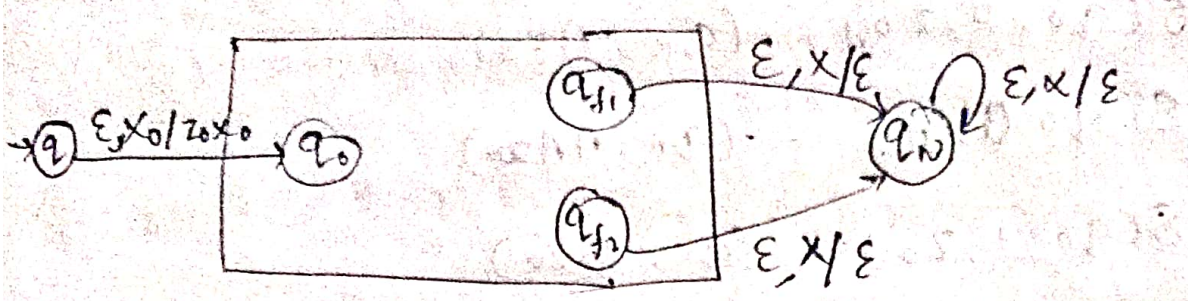
only 6-tuple because acc by empty stack.

→ Conversion of above eg: add $\delta(q, \varepsilon, x_0) = (q_0, z_0 x_0)$

$M_1 = \{q_0, q, q_f\}, \Sigma, \Gamma \cup \{x_0\}, \delta, q, x_0, q_f\}$ $\delta(q_0, \varepsilon, x_0) = (q_f, x_0)$

acceptance by final state, $q_f \in F$

2) Acceptance by final state to acceptance by empty stack

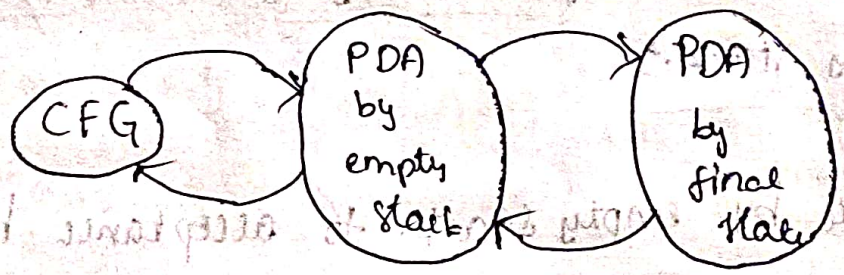


Eg → convertion again add

$$\delta(q_f, \varepsilon, X_0) = (q_N, \varepsilon)$$

$$\delta(q_N, \varepsilon, X) = (q_N, \varepsilon)$$

$$\delta(q_N, \varepsilon, X_0) = (q_N, \varepsilon)$$ → Acceptance by empty stack.



* Equivalence of CFG & PDA:

1) CFG to PDA
2) PDA to CFG (Assignment)

(1)

Let: $G = (V, T, P, S)$ be a CFG:

Construct a PDA P that accepts L(G) by empty stack

$P = \{\{q\}, T, V \cup T, \delta, q, S\}$ → start state of grammar is initial stack symbol

δ is defined as

1) For each non terminal A, ~~$\delta(q,\varepsilon$~~

$$\delta(q,\varepsilon,A) = \{(q,\beta) \mid A \to \beta \text{ is a P.R of } G\}$$

2) For each terminal a

$$\delta(q,a,a) = \{(q,\varepsilon)\}$$

Eg: Convert the following grammar into PDA.

$$E \to I / E*E / E+E / (E)$$
$$I \to a/b/Ia/Ib/I0/I1$$

if not specified LHS of 1st PR is start symbol S

$V = \{E, I\}$

$T = \{a, b, 0, 1, *, +, (,)\}$

$S = E$

$\delta(q,\varepsilon,E) = \{(q,I), (q,E*E), (q,E+E), (q,(E))\}$

$\delta(q,\varepsilon,I) = \{(q,a), (q,b), (q,Ia), (q,Ib), (q,I0), (q,I1)\}$

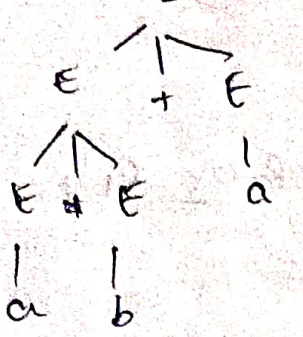
$\delta(q,a,a) = \{(q,\varepsilon)\}$ → $\delta(q,0,0) = \{(q,\varepsilon)\}$

$\delta(q,b,b) = \{(q,\varepsilon)\}$ → $\delta(q,1,1) = \{(q,\varepsilon)\}$

→ $\delta(q,*,*) = \{(q,\varepsilon)\}$ $\quad \delta(q,(,() = \{(q,\varepsilon)\}$

$\delta(q,+,+) = \{(q,\varepsilon)\}$ $\quad \delta(q,),)) = \{(q,\varepsilon)\}$.

→ a*b+a Es.



$\delta(q, a*b+a, E)$

$\vdash (q, a*b+a, E+E) \vdash (q, a*b+a, E*E+E)$

$\vdash (q, a*b+a, I*E+E) \vdash (q, a*b+a, a*E+E)$

$\vdash (q, *b+a, *E+E) \vdash (q, b+a, E+E)$

$\vdash (q, b+a, I+E) \vdash (q, b+a, b+E)$

$\vdash (q, +a, +E) \vdash (q, a, E) \vdash (q, a, I)$

$\vdash (q, a, a) \vdash (q, \varepsilon)$ → Accept by empty stack.

* Design PDA for the language

$L = \{a^i b^j c^k \mid i = j+k,\ i, j, k \geq 0\}$

push a's & b's then pop c's. → stack becomes empty & ε then accepted by PDA

$L = \{a^m b^n \mid n > m\}$

$L = \{a^n b^{2n}\}$

* Books > Examples

* Turing Machine (TM): 1936

Motivation: To capture the notion of computability.
↓
whether a problem can be solved?

* Claim:

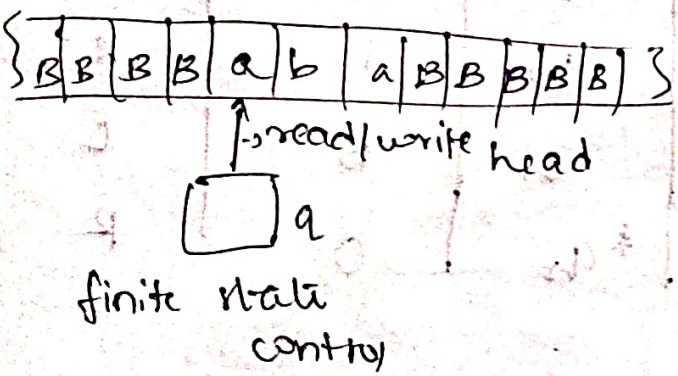
→ Whatever the computers in future or today cannot do what turing machine cannot do and whatever they can do turing machine can do.

* Church Turing thesis:

→ What can be done algorithmically can be done by TM & vice versa.

Formally:

→ There is an effective algorithm for solving a problem iff there is a TM that halts for all i/p & solve the problem.

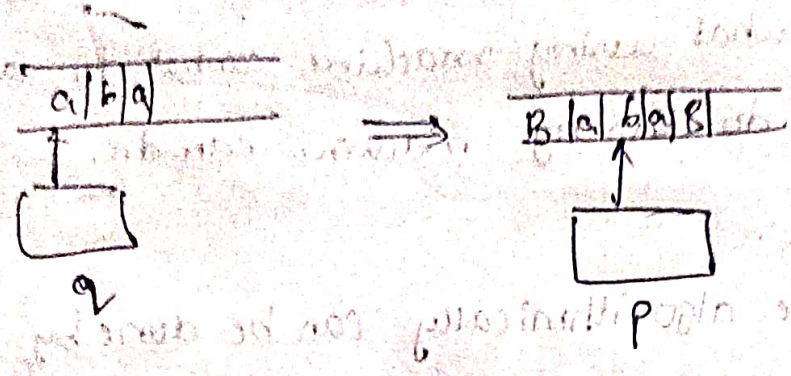


* 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, B, F)$

- Q: finite set of states
- Σ: set of input symbols, $\Sigma \subseteq \Gamma - \{B\}$
- Γ: set of tape symbols
- q_0: start state, $q_0 \in Q$
- B: Blank symbol, $B \in \Gamma$
- F: set of final states, $F \subseteq Q$

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$$

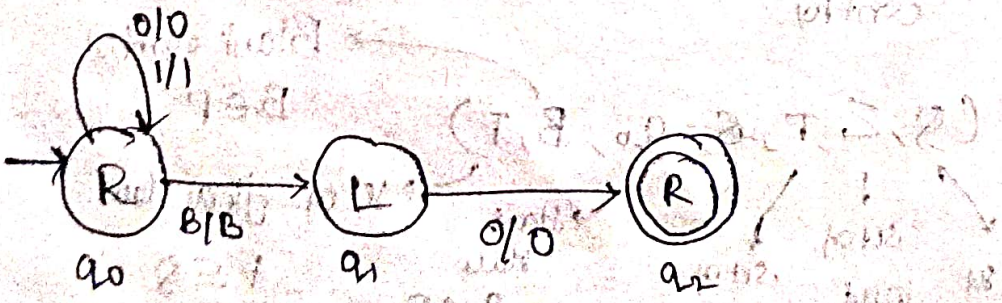
Current state	Scanned Symbol	Next State	Writing Symbol	Move
q	a	p	b	R

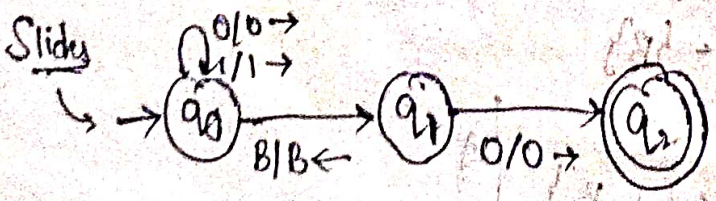


Eg: Design ~~turn~~ TM that accepts strings ending with zero.

$\Sigma = \{0, 1\}^*$

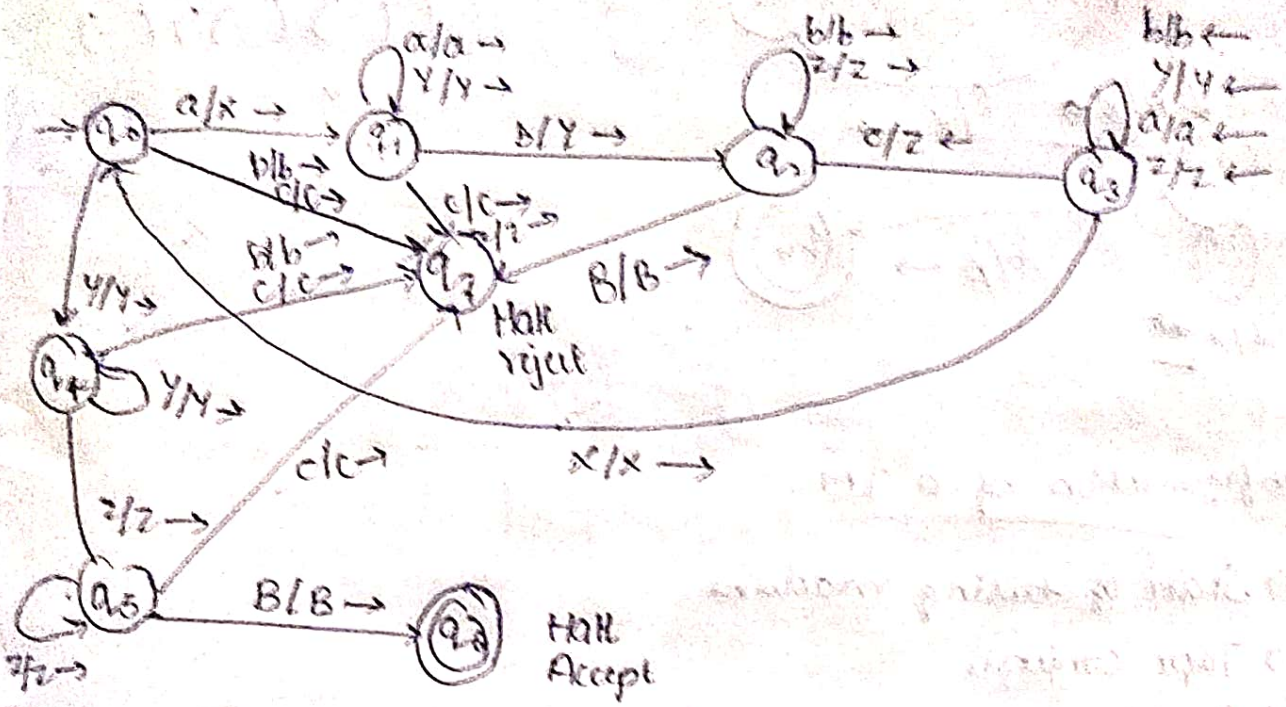
Current State	Scanned symbol	Next State	Writing symbol	Move
q_0	0	q_0	0	R
q_0	1	q_0	1	R
q_0	B	q_1	B	L
q_1	0	$*q_2$	0	R



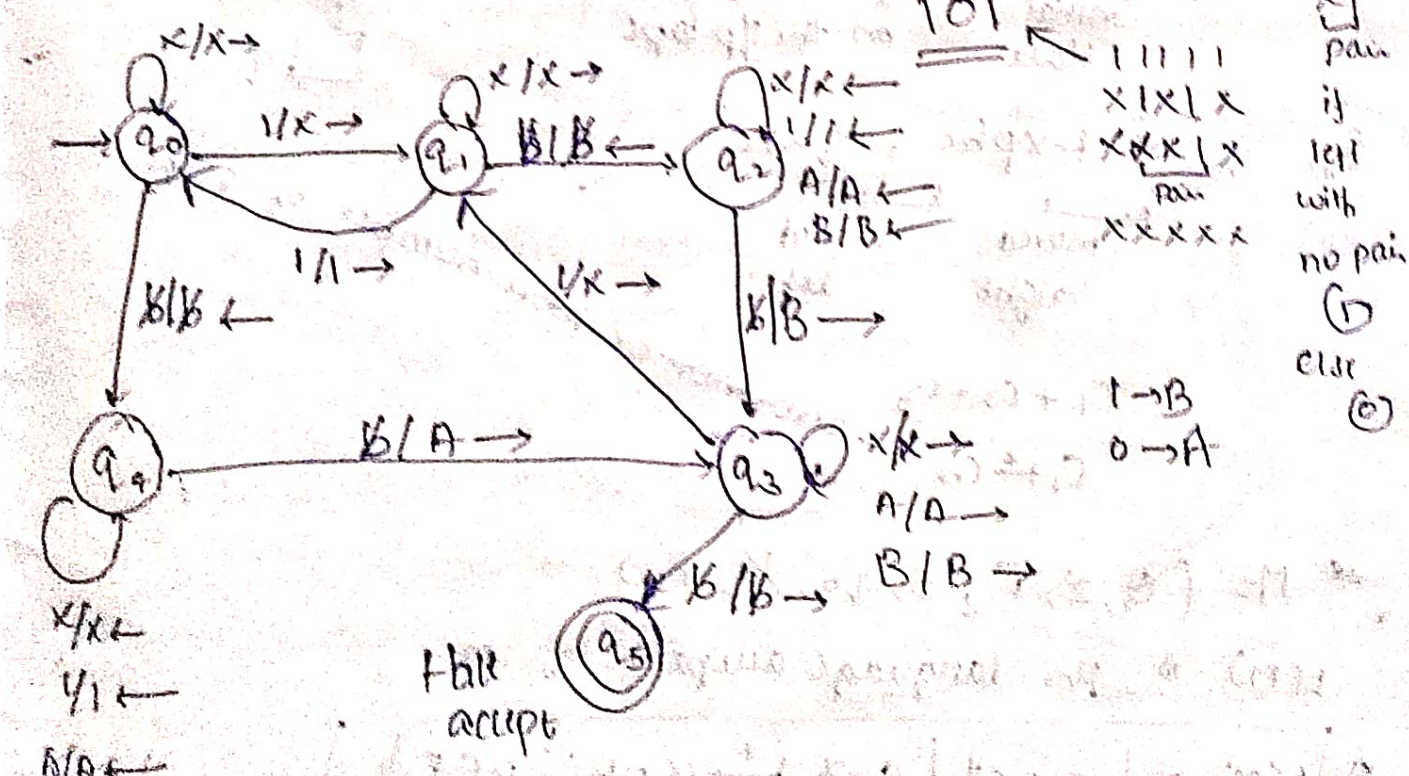


Eg: Design TM that accepts the following language.

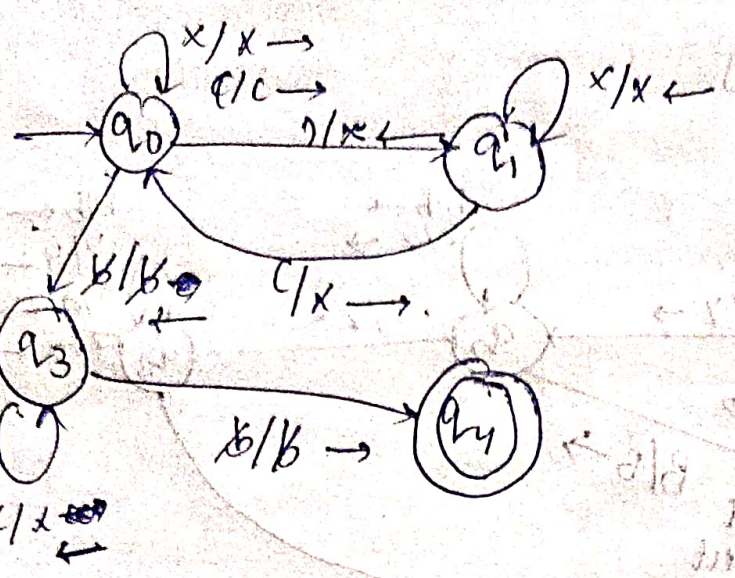
$L = \{a^n b^n c^n \mid n \geq 0\}$



Eg: Design a TM to convert unary number to binary number



Eg: Design a TM to accept the language of well formed parenthesis



(( ) ( )) ( )

x x x x x x x

* Configuration of a TM:

1) State of turing machine
2) Tape contents
3) Position of read/write head.

Initial configuration:

q_0x

↙ initial state

↘ leftmost symbol on the i/p tape

x = abc

b	b	a	b	c	b	b

↑ q

Eg: $q_0abc \vdash xpbc \vdash qxyc$

moved right

moved left

$C_1 \vdash C_2 \vdash C_3$

$C_1 \vdash^* C_n$ ← zero or more number of steps

* $M = (Q, \Sigma, \Gamma, \delta, q_0, b, F)$

L(M) is the language accepted by M;

$$L(M) = \{ x \in \Sigma^* \mid q_0x \vdash^* \alpha p \beta, p \in F \}$$

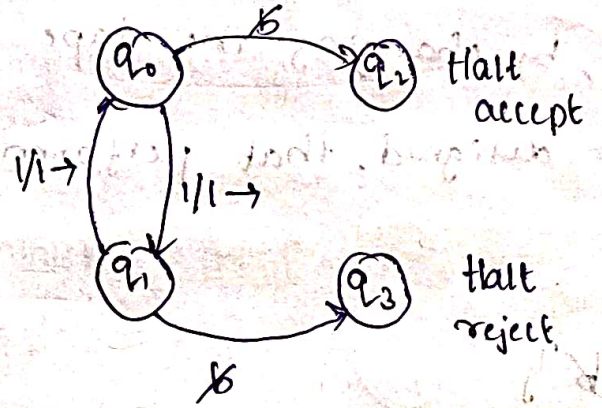
* Recursively enumerable language (R.E).

→ A language L is said to be RE if there is a TM M, such that $L(M) = L$.

Eg: we defined a TM for $L = \{a^n b^n c^n \mid n \geq 0\}$,

therefore $L = \{a^n b^n c^n \mid n \geq 0\}$ is a RE.

Eg: $L = \{1^n \mid n \geq 0$ & n is even$\}$



* Recursive languages (R) is a subset of RE

* If $L \in RE - R$ then $\bar{L} \notin RE$

* $L \in R \Leftrightarrow \bar{L} \in R$

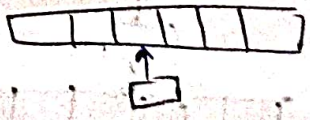
* $L \in RE$ and $\bar{L} \in RE$ $\Rightarrow L \in R$

* In TM, halt reject is not definitive.

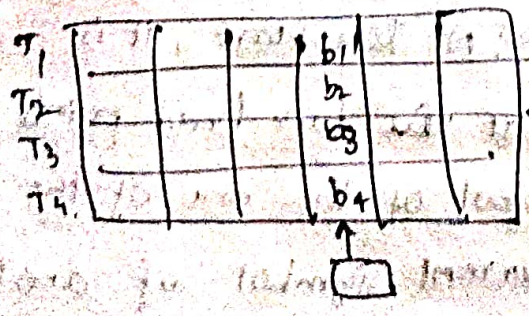
- In a TM
  - → Acceptance of a string is definitive
  - → Non-acceptance / reject of a string is non-definitive that is a TM may involve a non-terminating computation.

* Variants of TM:

Basic TM



1) Multitracking TM



Present state	i/p symbols	next state	output symbol	move
q	b_1	p	b_1'	R
	b_2		b_2'	
	b_3		b_3'	
	b_4		b_4'	

* We can reduce multi-track → basic TM.

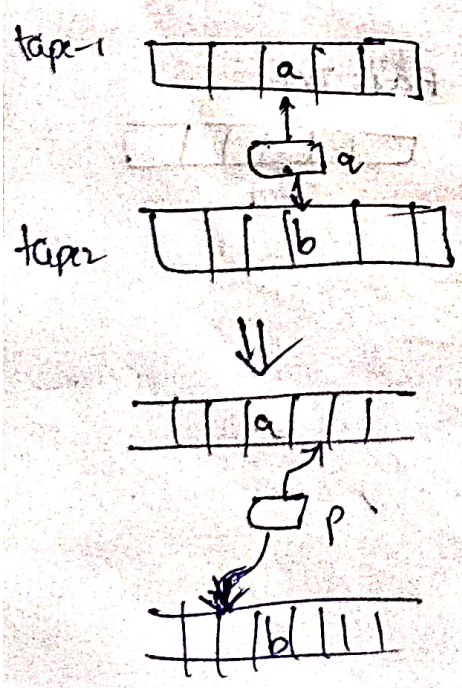
→ Therefore, if one could recognize a language by a multitrack TM, by changing the tape alphabet, a basic TM can be designed that performs same as multi-track TM.

Eg: $\Gamma = \{0, 1, b\}$

A	B	C	D	E	F	G	H	I
0	0	1	1	b	0	1	b	b
0	1	0	1	0	b	b	1	b

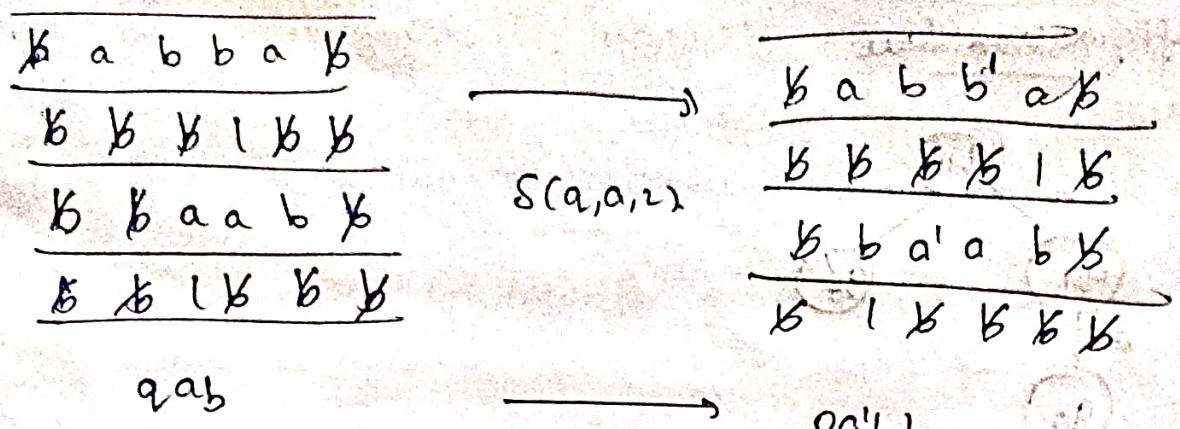
$$(q, \begin{matrix}0\\0\end{matrix}, p, \begin{matrix}\phi\\\phi\end{matrix}, R) \in \Rightarrow (q, A, p, I, R)$$

2) Multi-tape TM

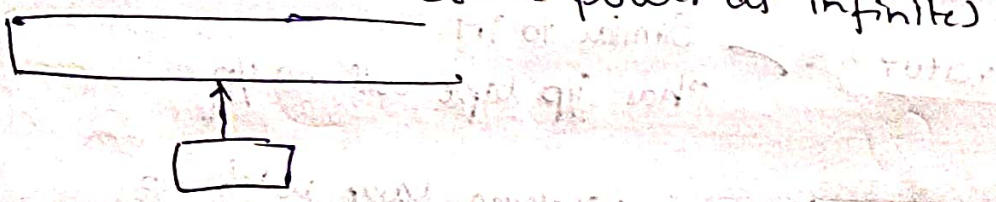


Present state	tape1 symbol	tape2 symbol	next state	tape1 move	tape2 move
q	a	b	p	a'	b'	R	L

Assumption: At the start, it is assumed that the tape head points to the current symbol of the one of the tapes and the current symbol of another tape, is positioned left of the current symbol.

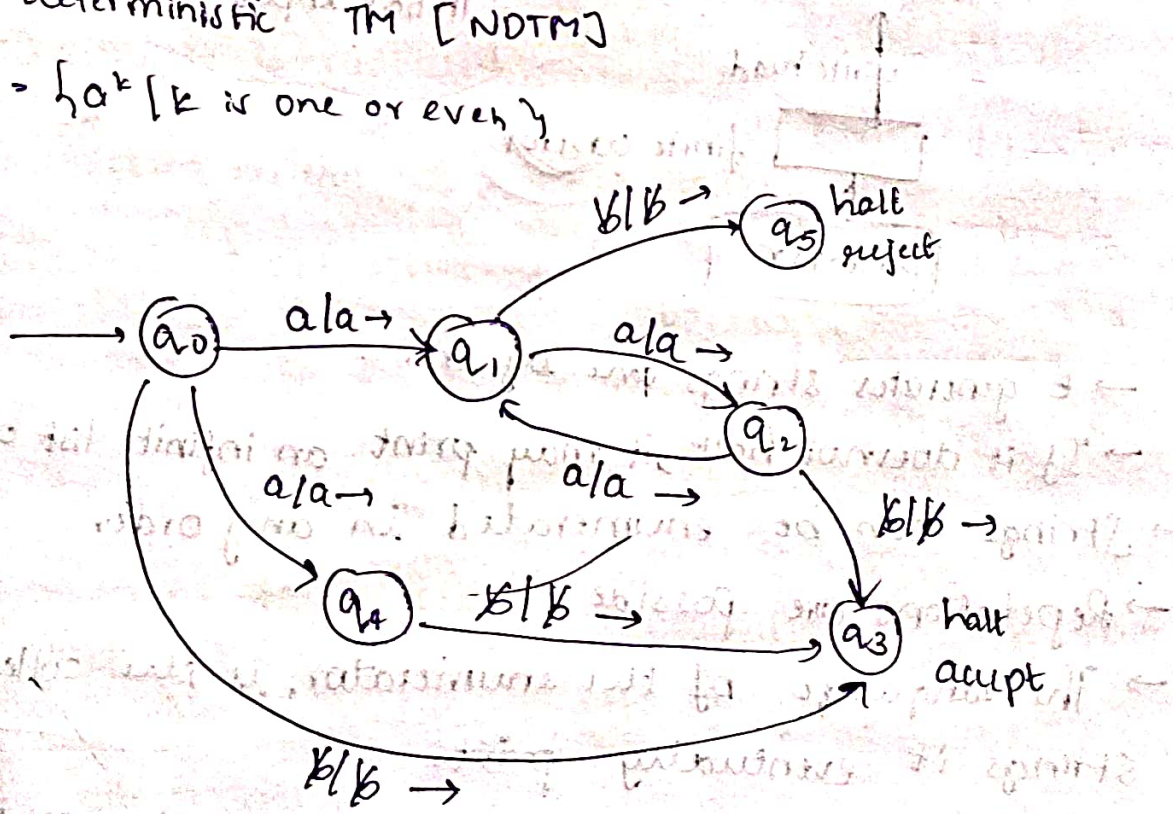


8) Semi infinite TM



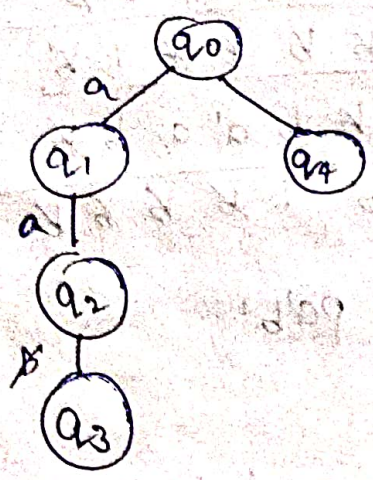
9) Non-deterministic TM [NDTM]

$L = \{a^k \mid k \text{ is one or even}\}$

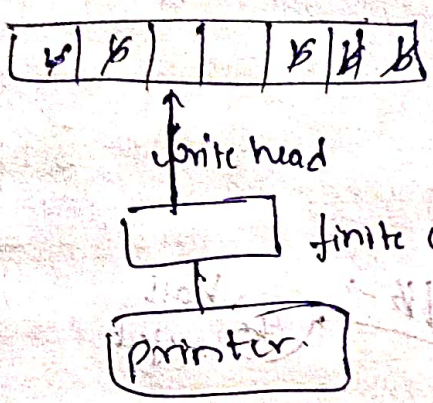


$L(M) \triangleq \{x \in \Sigma^* \mid q_0 x \vdash^* \alpha p \beta, p \in F\}$

Configuration tree



* Enumerator : → Similar to TM
→ has i/p tape with no i/p.



→ E generates strings one by one.
→ If it does not halt, it may print an infinite list of strings
→ Strings can be enumerated in any order
→ Repetetions are possible
→ The language of the enumerator, is the collection of strings it eventually prints.
→ Note that unlike TM it is not accepting or rejecting strings, it is generating strings.
→ We can assume that the enumerator E writes one string at a time over a tape(it can use a tape symbol # to separate strings)

* Theorem:

A language is Turing recognizable. → R.E.L; that is accepted by a TM.

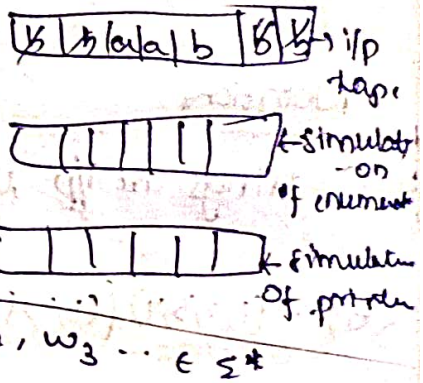
if & only if some enumerator enumerates it.

Proof:

1) Simulating an enumerator on a TM M.

M = for i/p w
  Simulate E (on blank i/p)
  whenever E prints x, check w=x
  Accept if w=x, otherwise continue.

TM M should accept/reject a string iff there is an enumerator that prints the string



2) Convert TM M to equivalent enumerator E.

E = Simulate M on each w_i in Σ^*
  whenever M accepts, print w_i

However on some i/p TM may go into infinite loop.

optimized soln

E = Repeat the following for $i = 1, 2, 3 \ldots$
  Run M for i steps on each input $w_1, w_2, \ldots w_i$
  if M accepts, print corresponding w_j

* Hilbert's 10th problem:

Diophantine equations:

→ Equations of polynomials where solns must be integers

Eg. $3x^2 - 2xy - y^2z = 7$

$x = 1$

$y = 2$

$z = -2$

Decidability of algorithms

* Given a problem there exist an algorithm, that solves (may accept / may reject) the problem on all inputs.

* A TM has 3 possible outcomes for each i/p w.

T-D { 1) Accept w (Enter q_{acc} - accept state & halt)
      2) Reject w (Enter q_{rej} - reject state & halt)

T-R { 3) Reject w by looping (Running forever) → ∞ loop

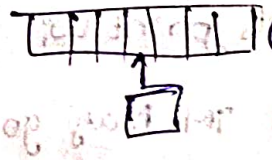
* A is T-recognizible if A = L(M) for some TM M. (non determinate)

* A is T-decidable if A = L(M) for some TM decider M. (halts on all inputs)

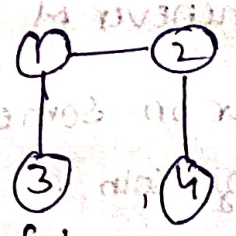
* Notations:

→ Strings are i/p to TM.

$\langle G \rangle = (1,2,3,4)\,(1,2)(1,3)(2,4))$



G =



M = "on input $\langle G \rangle$, the encoding of graph G:

1) Select the first node & mark it

2) Repeat until no new nodes are marked. For each node in G mark it if there is an edge attaching it to an already marked node.

3) Scan all nodes in G, if all are marked, then accept else reject.

1) Acceptance problems for DFAs.

Let $A_{DFA} = \{ \langle B, w \rangle \mid B \text{ is a DFA \& } B \text{ accepts } w \}$

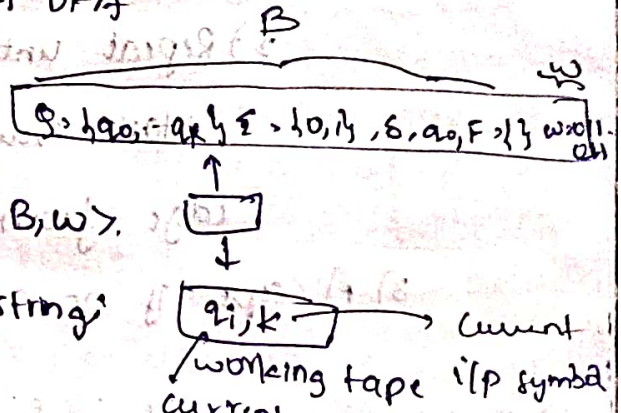
Theorem: A_{DFA} is decidable

Proof: Give a TM $D_{A\text{-}DFA}$ that decide A_{DFA}.

decidable TM for the problem of acceptance of DFA

$D_{A\text{-}DFA}$ = "on input string s

1) check that s has the form $\langle B, w \rangle$. where B is DFA & w is a string; reject if not



2) Simulate the computation of B on w.

3) If B ends in accept state then accept, if not then reject

2) Acceptance problem for NFAs:

Let $A_{NFA} = \{ \langle B, w \rangle \mid B \text{ is a NFA \& } B \text{ accepts } w \}$

Theorem: A_{NFA} is decidable

Proof: Given a TM $D_{A\text{-}NFA}$ that decide A_{NFA}

$D_{A\text{-}DFA}$ = "on i/p $\langle B, w \rangle$

1) Convert NFA B to equivalent DFA B'.

2) Run TM $D_{A\text{-}DFA}$ on input $\langle B', w \rangle$ → subroutine from prev prob

3) Accept if $D_{A\text{-}DFA}$ accepts, reject if not.

3) Emptiness problem for DFAs.

let $E_{DFA} = \{ \langle B \rangle \mid B \text{ is a DFA } \& \ L(B) = \emptyset \}$.

Theorem: E_{DFA} is decidable

Proof: Give TM $D_{E\text{-}DFA}$ that decides E_{DFA}.

$D_{E\text{-}DFA}$ = "on input $\langle B \rangle$

1) mark the start state
2) Repeat until no new state is marked.
   Mark every state that has an incoming edge from a previously marked state.
3) Accept if no accept state is marked, else reject.

4) Equivalence problem for DFAs.

let $EQ_{DFA} = \{ \langle A, B \rangle \mid A \ \& \ B \text{ are DFAs } \& \ L(A) = L(B) \}$.

Theorem: EQ_{DFA} is decidable.

Proof: Give TM $D_{EQ\text{-}DFA}$ that decides EQ_{DFA}

$D_{EQ\text{-}DFA}$ = "on input $\langle A, B \rangle$

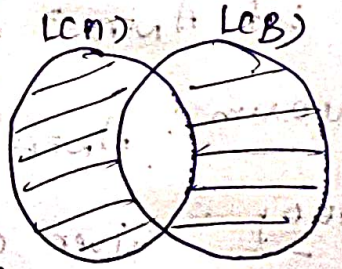
1. Construct DFA C where

$$L(C) = (L(A) \cap \overline{L(B)}) \cup (L(B) \cap \overline{L(A)})$$

2) Run $D_{E\text{-}DFA}$ on $\langle C \rangle$.

3) Accept if $D_{E\text{-}DFA}$ accepts else reject.

→ it has strings rejected by either one of A or B.



symmetric diff

if symmetric diff is empty, then L(A) = L(B)

5) Equivalence problem for Regular expressions:

Let $EQ_{REX} = \{\langle R_1, R_2 \rangle \mid R_1 \text{ \& } R_2 \text{ are REs \& } L(R_1) = L(R_2)\}$.

Theorem : EQ_{REX} is decidable.

6) ~~Equivalence~~ Acceptance problem for CFG

Let $A_{CFG} = \{\langle G, w \rangle \mid G \text{ is a CFG \& } w \in L(G)\}$

Theorem : A_{CFG} is decidable.

Proof : " on input $\langle G, w \rangle$

1) Convert G into CNF. → efficient way (if not in CNF also CGF)
2) Try all derivations of length $2|w|-1$ is decidable
3) Accept if any derivation generates w, reject if not.

6.1) Every CFL is decidable.

Proof : Let A be a CFL generated by CFG G.

Construct TM M_G = " on input w

1) Run D_{A-CFG} on $\langle G, w \rangle$
2) Accept if D_{A-CFG} accepts, reject if it rejects.

7) Acceptance problems for PDA

Theorem : A_{PDA} is decidable

As all PDAs can be converted to CFG, we already know acceptance of CFGs is decidable, therefore acceptance for PDA is decidable.

8) Emptiness problem for CFG.

Let $E_{CFG} = \{\langle G \rangle \mid G \text{ is a CFG \& } L(G) = \phi\}$

Theorem: E_{CFG} is decidable.

Proof: $D_{E\text{-}CFG}$ = " on input $\langle G \rangle$

1) Mark all the occurrences of terminals in G.

2) Repeat until no new variables are marked. Mark all occurrences of a variable A if $A \to B_1 B_2 \cdots B_k$ is a PR & all B_i were marked.

3) Reject if the start symbol is marked else accept.

9) Equivalence problem for CFGs

Let $EQ_{CFG} = \{\langle G, H \rangle \mid G, H \text{ are CFGs \& } L(G) = L(H)\}$

Theorem: EQ_{CFG} is not decidable.

10) Ambigous problems for CFGs.

Let $AMBIG_{CFG} = \{\langle G \rangle \mid G \text{ is an ambiguous CFG.}\}$

Theorem: $AMBIG_{CFG}$ is not decidable.

11) Acceptance problem for Turing machines:

Let $A_{TM} = \{\langle M, w \rangle \mid M \text{ is a TM \& } M \text{ accepts } w\}$

Theorem: A_{TM} is ~~T-recognizable~~ & not decidable but T-recognizible.

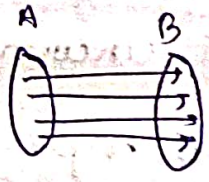
Proof: The following TM U recognizes A_{TM}.

U = "On i/p $\langle M, w \rangle$.

1) Simulate M on i/p w
2) Accept if M halts & accepts
3) Reject if M halts & rejects.
4) ~~Reject if M never halts.~~ " → Not a legal TM move.

→ How to compare relative sizes of sets of infinity.

Say that there are two sets A & B, such that $|A| = |B|$ iff there is a one to one (injective) and onto (surjective) fn $f: A \to B$



range of f is all elements in B

N → set of natural numbers. $\{1, 2, 3, \ldots\}$

Z → set of integers. $\{\ldots -2, -1, 0, 1, 2, \ldots\}$

n	f(n)
1	0
2	-1
3	1
4	-2
5	2
⋮	⋮

$Q^+ = \{m/n \mid m, n \in N\}$

Q^+	1	2	3	4
1	1/1	1/2	1/3	1/4
2	2/1	2/2	2/3	2/4
3	3/1	3/2	3/3	3/4
4	4/1	4/2	4/3	4/4
⋮	⋮	⋮	⋮	⋮

□ → if written one one relation fails

* Every CFL is decidable

N^n	$f(n)$ $\mathbb{Q}^+$
1	1/1
2	2/1
3	1/2
4	3/1
5	3/2
6	2/3
7	1/3
8	
9	

* A set is countable if it is finite or it has same size as $\mathbb{N}$

Eg: $\mathbb{Z}$, $\mathbb{Q}^+$

• Does all infinite sets have same size as $\mathbb{N}$?

No ($\mathbb{R}$)
↳ real numbers

Statement: * Set of real numbers is uncountable

Let $\mathbb{R}$ is set of real numbers represented as infinite decimal expansion

Theorem: $\mathbb{R}$ is uncountable

Proof by contradiction: Let $\mathbb{R}$ is countable, then $\exists f$ which is one-one & onto.

n	f(n)
1	0.55822910 - -
2	0.61341523 ——
3	0.72341156 - —
4	0.91245789 ——

$X = 0.6276 - —$

if we compare x with some ith number then x is different ith number at ith position.

So $|R| > |N|$

Let L = all languages that is every subset of Σ^*

Corollary: L is uncountable.

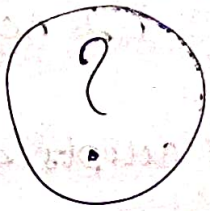
Proof: There is a 1-1 correspondence from L to R.

So they are uncountable.

$\Sigma^* = \{\varepsilon, 0, 1, 01, 10, 11, 000, 111, 1011, \ldots\}$

$A = \{1, 01, 11, 111, 1011, \ldots\}$

$B = \{0, 10, 000, \ldots\}$



* Σ^* is countable, L is uncountable

Let M = set of TMs.

M is countable, because

$\{\langle M \rangle, |M \text{ is a TM}\} \subseteq \Sigma^*$

Corollary 2: Some languages are not decidable.

because there are more languages for which there is no TM.

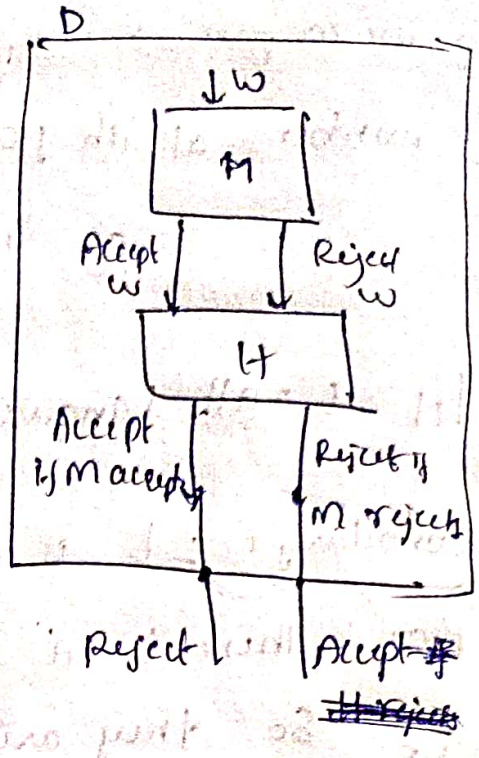
also T-recognizable

Theorem: A_{TM} is not decidable

Proof by contradiction:

Assume some TM H decides A_{TM}.

So H on $\langle M, w \rangle$ = { accept if M accept w; ~~Accept~~ reject if M reject w



Use H to construct TM D.

$D_=$ on input $\langle M \rangle$

1. Simulate H on input $\langle M, \langle M \rangle \rangle$
2. Accept if H rejects. Reject if H accepts.

* D accepts $\langle M \rangle$ iff M doesn't accept $\langle M \rangle$.
* D accepts $\langle D \rangle$ iff D doesn't accept $\langle D \rangle$.

not a valid statement.

So one of our assumption is false so A_{TM} is not decidable

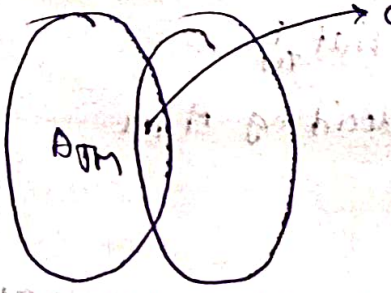
	$\langle M_1 \rangle$ $\langle M_2 \rangle$ $\langle M_3 \rangle$ ...	$\langle D \rangle$
M_1	acc rej rej ...	
M_2	rej rej rej ...	...
M_3	acc acc rej ...	...
.		
.		
.		
D	rej acc acc ...	?

no. of machines is not equal to machine descriptions.

$\overline{A_{TM}}$ is T-unrecognizable.

→ A language is co-Turing recognizable if it is the complement (complement) of Turing-recognizable language.

Theorem: A language is decidable if it is T-recognizable and co-Turing-recognizable



T-recog Complement of T-recog

Are there any languages which are neither RE nor Co-RE?

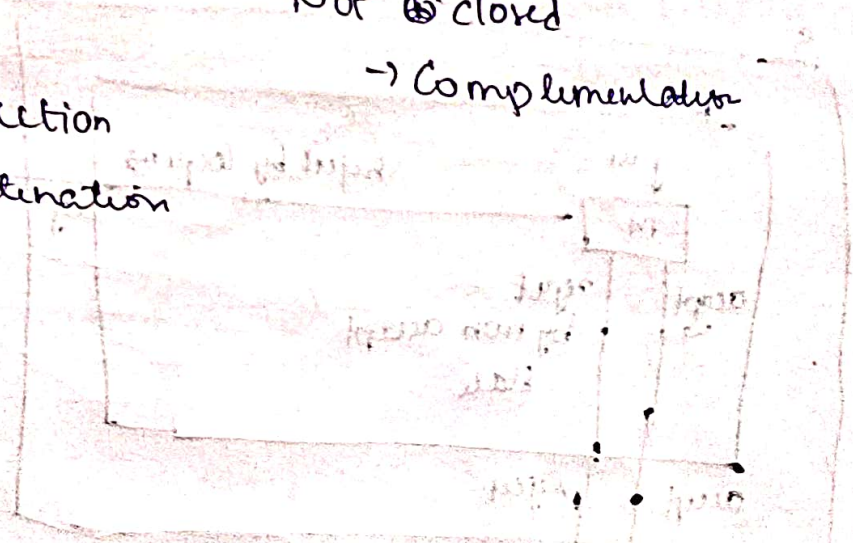
Yes.

$EQ_{TM} = \{\langle M_1, M_2\rangle \mid L(M_1) = L(M_2)\}$

* Closure properties of T-recognizable

Closed
- → union
- → Intersection
- → Concatenation
- → Star.

Not closed
- → Complementation



* Reducibility:

Halting problem:

→ given an i/p w whether a TM halts?

Statement: Halting problem is undecidable.

$Halt_{TM} = \{\langle M, w\rangle \mid M \text{ halts on } w\}$

Theorem: $Halt_{TM}$ is undecidable.

$A_{TM} = \{\langle M, w\rangle \mid M \text{ accepts } w\}$.

~~Let TM R decides~~

Let us reduce A_{TM} to $Halt_{TM}$.

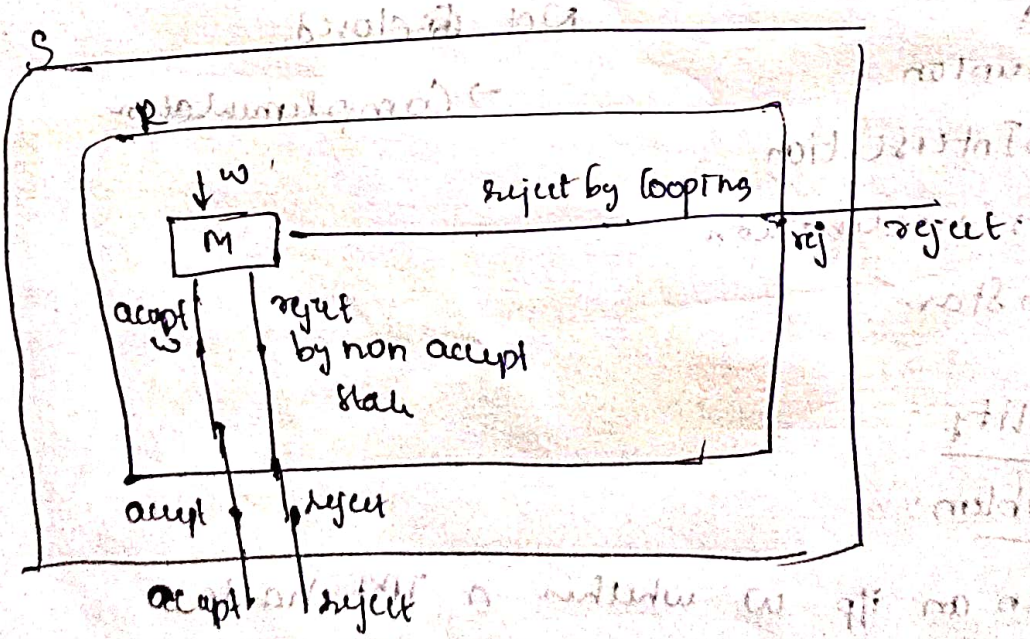
Assume that $Halt_{TM}$ is decidable & show that A_{TM} is decidable (false).

Let TM R decide $Halt_{TM}$.

Construct TM S deciding A_{TM}

S = on i/p $\langle M, w\rangle$

1. Use R to test if M on w halts. If not ~~reject~~ reject.
2. Simulate M on w until it halts.
3. If M has accepted then accept
   if M has rejected then reject



* If $A \leq_m B$ (reducible), then solving B gives a soln to A

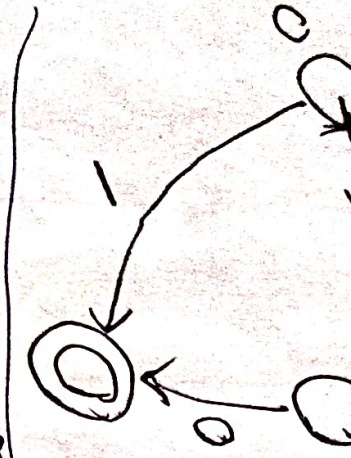
area ↙ length & breadth

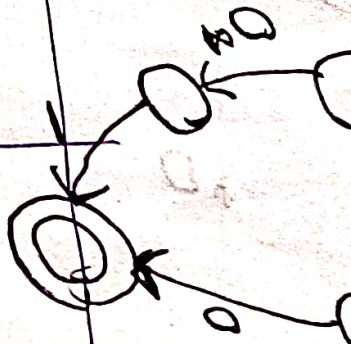
→ then B is easy, A is easy ✓

→ then A is hard, B is hard

A is undecidable → B is undecidable

A_{TM} is undecidable → $Halt_{TM}$ is undecidable





Problem A reduced to B

↓

If B is decidable then

A is decidable

↓

If A is undecidable then

B is undecidable